Cover Page

Student Name: Yuhan Feng

High School: Zhuzhou Nanfang High School

Province: Hunan

Country/Region: China

Advisor Name: Shu Shi, Li Zixuan

Advisor Affiliation: Zhuzhou Nanfang High School

Paper Title: Selective Belief Propagation for Efficient Truth Maintenance in LLM Agents

Selective Belief Propagation for Efficient Truth Maintenance in LLM Agents

Yuhan Feng

Abstract

LLM agents operating over extended sessions maintain a dependency graph of facts and derived conclusions; when a fact is updated, a natural baseline for update propagation is to mark *every* reachable downstream conclusion with a binary STALE flag, triggering full recomputation regardless of the update's magnitude.

We address this problem by proposing **Neural Truth Maintenance** (NTM), a framework that replaces the binary STALE flag with a continuous $\mathcal{B}(f) \in [0, 1]$ belief state and propagates confidence decay—rather than invalidation—along the edges of a probabilistic dependency graph. Here, "neural" refers to the semantic gate implemented by an LLM, rather than to a learned differentiable propagation operator. **NTM-BP** is a propagation engine that multiplicatively decays belief along graph edges using a saturating source-change scaling and a propagation decay coefficient γ, marking only conclusions whose belief drops below a re-evaluation threshold τ. The **Structured Semantic Gate** decomposes the gate-triggering decision into three discrete LLM judgments—change type (A/B/C), relevance (counterfactual), and stance (aligned/opposed)—so that each update takes the cheapest sufficient path: minor fluctuations terminate after a single call, while ambiguous cases receive deeper semantic analysis. The two mechanisms are complementary: NTM-BP provides a zero-cost numeric backbone; the structured gate adds semantic precision where numeric decay alone is insufficient.

On a post-freeze held-out validation set of 54 never-seen cases (dual-annotated, $\kappa = 1.0$; only 17 with non-empty ground truth), the structured gate attains the best point estimate among the evaluated non-lexical methods (F1 = 0.778 versus 0.686 for NTM-BP and 0.315 for BFS), but at $n = 54$ this gain over NTM-BP is *not* statistically significant at $\alpha = 0.05$ ($p = 0.18$); we therefore position the structured gate's contribution as diagnosability and early-exit cost savings rather than a demonstrated accuracy win, and read the headline number as a small-sample point estimate. NTM-BP alone reduces the re-evaluation rate by 29.3% versus BFS while maintaining F1 = 0.818 on the full PC-500 set. We honestly report a zero-cost keyword baseline reaching F1 = 0.944 and characterize it as a lexical boundary condition, and identify supersede-strengthening misclassification as the structured gate's primary observed failure mode on the 54-case held-out expansion set. On a second, *balanced adversarial* held-out set (40 cases, 20/20 non-empty/empty) constructed so that lexical and confidence cues are uninformative, the structured gate significantly outperforms both NTM-BP (+0.110, $p < 0.001$) and a zero-cost keyword rule (+0.250, $p = 0.001$), locating the structured gate's value to the regime where update impact is not lexically marked.

Keywords: Belief Propagation, Truth Maintenance, LLM Agent Memory, Dependency Graph, Selective Re-evaluation

Contents

1 Introduction **1**

2 Related Work **3**
2.1 Classical Truth Maintenance 3
2.2 AGM Belief Revision 4
2.3 LLM Agent Memory Systems 4
2.4 Belief Propagation and Causal Inference 5
2.5 Semantic Gate Mechanisms 6

3 Method **7**
3.1 Problem Formulation 7
3.2 NTM-BP: Numeric Belief Propagation 7
3.3 Structured Semantic Gate 8

4 Theoretical Analysis **10**
4.1 Cascade Completeness (Baseline) . . . 10
4.2 NTM-BP Convergence on Trees . . . 11
4.3 Precision-Efficiency Tradeoff . . . 11
4.4 Bounded Belief Perturbation . . . 11

5 Experiments **12**
5.1 RQ1: Is NTM Effective? . . . 13
5.2 Balanced Adversarial Held-out Set . . . 15
5.3 RQ2: Why Is NTM Effective? . . . 16
5.4 RQ3: Which Structured Gate Components Contribute? . . . 16
5.5 RQ4: Where Does the Method Fail? . . . 17

6 Discussion **20**
6.1 Why the Structured Gate Works . . . 20
6.2 Design Principles for NTM Gates . . . 23
6.3 Limitations and Future Work . . . 24

7 Conclusion **26**

A Reproducibility and Audit Protocol **33**

B Full Gate Prompt Texts **34**

C Formal Definitions and Proofs **35**
C.1 Formal Memory Definitions . . . 35
C.2 Proofs and Extended Derivations . . . 36

1 Introduction

Large language model (LLM) agents that operate across hundreds of dialogue turns accumulate a growing set of inferred conclusions—*beliefs*—that rest on a smaller set of observed or asserted *facts* [6, 7, 9, 16, 20]. These facts and beliefs form a *dependency graph*: a directed acyclic graph where an edge $f_i \to f_j$ indicates that conclusion f_j was derived from or depends on fact f_i. When a fact changes (e.g., "Acme Corp reputation is strong" is updated to "Acme Corp is under investigation"), the agent must decide which downstream conclusions remain valid and which need to be re-derived.

A natural baseline for update propagation, related to copy-on-write (CoW) versioning and classical Truth Maintenance Systems (TMS) [2, 3], uses a *binary STALE flag*: every conclusion that transitively depends on the updated fact is marked *stale* (assigned belief $\mathcal{B} = 0$) and unconditionally recomputed. A breadth-first cascade traversal propagates this flag in $O(|V| + |E|)$ time with zero semantic cost, but at the price of 100% downstream recomputation—including conclusions whose relationship to the updated fact is weak, orthogonal, or purely administrative. We refer to this binary propagation behavior as the *BFS cascade*.

This "all-or-nothing" cascade is the right behavior when a fact is *fully contradicted* (e.g., a supplier that went bankrupt). However, many real-world updates are *partial*: a situation worsens but does not reverse, a minor fluctuation occurs, or a previously unclear detail is merely confirmed. Marking all downstream conclusions as STALE when a single-day traffic spike barely differs from the trend is analogous to discarding every file on a disk when one sector changes. Worse, for agents embedded in cost-sensitive deployment pipelines, each unnecessary re-evaluation consumes an LLM call.

Core idea. We propose to replace the binary STALE flag with a *continuous belief state* $\mathcal{B}(f) \in [0, 1]$—a confidence score assigned to each fact—and to propagate confidence decay along the edges of a probabilistic dependency graph, where each edge carries an edge dependency weight $w_{ij} = P(f_j \mid f_i)$. This turns the gate-triggering problem from "should I re-evaluate everything?" into "which conclusions need re-evaluation *this time*?". We call this framework **Neural Truth Maintenance** (NTM)—where "neural" denotes LLM-assisted semantic reasoning, not a learned differentiable propagation operator—situating it as a probabilistic

generalization of Doyle's classical TMS [2]: where TMS uses symbolic IN/OUT labels, NTM uses continuous $[0, 1]$ belief scores; where TMS propagates retraction, NTM propagates confidence decay.

Two complementary mechanisms. NTM is realized through two mechanisms that operate at different levels:

1. **NTM-BP** (numeric): A belief-propagation engine that multiplicatively decays belief along graph edges, using a saturating source-change scaling $\alpha(\Delta)$ and a propagation decay coefficient γ. It is purely numeric, requires no LLM calls, and runs in $O((|V| + |E|) \log |V|)$ time on tree-structured graphs with a binary-heap priority queue (the underlying traversal is linear in the number of visited nodes and edges). NTM-BP serves as the zero-cost numeric backbone of the framework.

2. **Structured Semantic Gate** (LLM): A decomposition of the gate decision into three discrete LLM judgments—change type (A/B/C), relevance (counterfactual), and stance (aligned/opposed)—so that minor fluctuations terminate after a single classification call, while ambiguous cases receive deeper semantic analysis. The structured gate adds semantic precision where NTM-BP's numeric decay alone is insufficient (e.g., distinguishing confirming updates from truly neutral ones).

The two mechanisms are complementary: NTM-BP provides a zero-LLM-cost fallback with F1 = 0.818 on the full PC-500 set; the structured gate adds LLM-based semantic analysis for higher precision at a modest call cost (average 2.87 calls per update).

Experimental discipline. Prompt-design choices strongly affect model behavior [12], and iterative prompt tuning risks overfitting to whichever cases the developer inspects. We therefore adopt a strict *held-out regime*: all prompt iterations are performed on a sealed development set (29 template-held-out cases: 9 from PC-500, 12 from SynDep, and 8 from the CO-Relation design set); the test data—including a 54-case post-freeze held-out set (Expansion-500) with dual-annotator labels—are generated and annotated *after* prompt freezing. Every LLM call is logged with a prompt-hash audit trail, and evaluation scripts refuse to overwrite results. A full reproducibility statement—the exact prompts, hyperparameters, seeds, evaluation protocol, and the self-contained reproducibility package—is given in Sec. A.

What we do and do not claim. Our central positive result is the zero-cost numeric engine NTM-BP, which cuts the re-evaluation rate by 29.3% versus the binary cascade while holding F1 = 0.818 on PC-500. On the original held-out expansion set, the structured gate's accuracy gain over NTM-BP is a *point estimate that is not statistically significant* at $n = 54$ ($p = 0.18$), and—because that set is dominated by empty ground truth—a zero-cost keyword baseline reaches F1= 0.944, *above* every method we propose; we report both prominently. This keyword advantage is, however, a property of the distribution rather than of the gate: when impact is not lexically marked and confidence changes are small, the rule collapses. On a second, *balanced adversarial* held-out set designed for exactly this regime (Sec. 5.2), the structured gate is *significantly* more accurate than both NTM-BP (+0.110, $p < 0.001$) and the keyword rule (+0.250, $p = 0.001$), while remaining statistically tied with the scalar gate. Our claims, positive and negative, are therefore scoped to the measured regimes, and we treat this scoping—not a headline number—as the paper's honesty commitment (Sec. 5).

Contributions.

- **New problem formulation.** We formalize downstream belief maintenance under fact updates as a continuous impact propagation problem: given a fact update and a probabilistic dependency graph, determine which downstream conclusions require re-evaluation based on propagated belief decay rather than binary invalidation (Sec. 3.1).

- **New method (NTM-BP).** We propose NTM-BP, a numeric belief-propagation engine that propagates update impact through dependency graphs using continuous belief scores, with saturating source-change scaling, multiplicative edge decay, and theoretical convergence guarantees on tree-structured graphs (Secs. 3 and 4).

- **New method (Structured Semantic Gate).** We design a structured LLM gate that decomposes the re-evaluation decision into three independently auditable sub-judgments (change type, relevance, stance), enabling cost-sensitive early exit and isolating failures to individual sub-questions (Sec. 3.3).

- **New experimental findings.** A multi-dataset held-out evaluation (cluster-bootstrap CIs, dual-annotator labels $\kappa = 1.0$—an annotation-policy consistency check rather than independent external

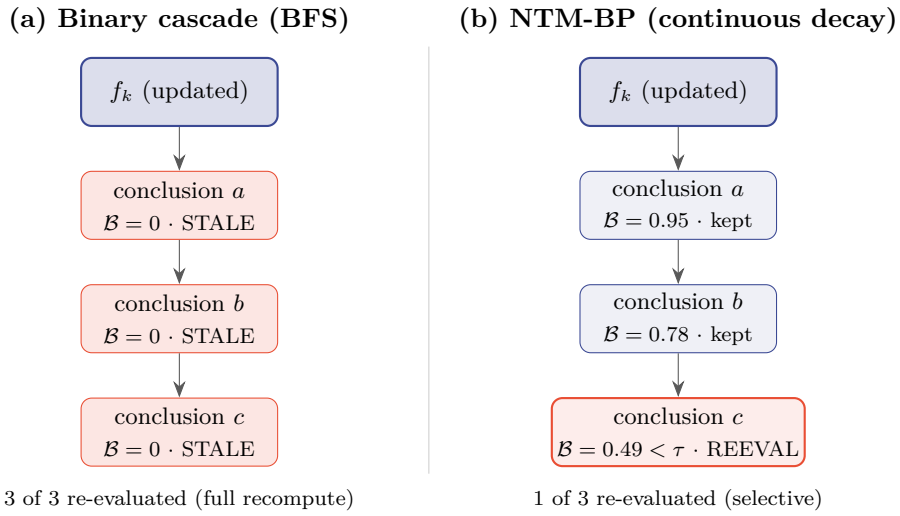


Figure 1: The core idea, on a four-fact dependency chain. **(a)** The binary cascade marks every reachable conclusion `STALE` and recomputes all of them. **(b)** NTM-BP instead propagates a continuous belief decay along the edges and marks only conclusions whose belief drops below the threshold τ (here $0.49 < \tau = 0.7$), leaving weakly affected conclusions intact. Values are illustrative.

validation, as the second annotator is an in-school advisor—component-frozen reproducibility) whose key results—including two negative ones we report in full, a failing routing strategy and a zero-cost keyword baseline that *outperforms* our gate—are detailed in Sec. 5.

Fig. 1 contrasts the two propagation strategies on a four-fact dependency chain: the binary cascade invalidates every reachable conclusion, whereas NTM-BP attenuates belief along the edges and marks only those whose belief falls below the threshold.

2 Related Work

2.1 Classical Truth Maintenance

Doyle [2] introduced the Truth Maintenance System as a mechanism to maintain consistency among a set of beliefs connected by justification links. When a supporting premise is retracted, the system propagates an OUT label to dependent conclusions. In this knowledge representation method, derived information (a formula) is explicitly stored and associated with the information it originates from by means of justifications. This dependency can be used to explain the reason for a conclusion (consequence) by its initial premises. With the help of justifications, conclusions can be withdrawn by retracting their premises. By this logical retraction, e.g. default reasoning can be supported and inconsistencies can be repaired by retracting one of the reasons for the inconsistency. An obvious application of justifications are dynamic constraint satisfaction problems (DCSP), in particular over-constrained ones [27]. De Kleer [3] generalized this to assumption-based TMS (ATMS), where each conclusion is annotated with the set of assumptions under which it holds. Both systems operate on *symbolic* truth values (IN/OUT). NTM retains the dependency-graph structure but replaces binary labels with $[0, 1]$ belief scores, making the framework compatible with the probabilistic reasoning that underlies modern LLM agents.

2.2 AGM Belief Revision

Alchourrón, Gärdenfors, and Makinson [4] axiomatized rational belief change through three postulates: consistency (the revised knowledge base must be consistent), informational economy (retain as much old knowledge as possible), and minimal change. NTM's copy-on-write versioning preserves prior versions (version availability, not logical consistency); the multiplicative decay mechanism operationalizes informational

economy by only downgrading (rather than discarding) beliefs that remain plausible; and the threshold comparison provides a bounded-perturbation property in a continuous setting by marking only sub-threshold nodes—a relaxation of the AGM postulate, since continuous belief decay intrinsically changes more belief values than a binary toggle. We view this as a pragmatically useful extension rather than a strict axiomatic derivation. D. Fazio and M. Pra Baldi investigate AGM belief contraction operators by using the tools of algebraic logic; they generalize the notion of contraction to arbitrary finitary propositional logics and show how to switch from a syntactic-based approach to a semantic one [28]. Park's Kumiho system [10] formalizes belief-revision semantics for versioned agent memory within an AGM-style framework; NTM is constructive in a different sense—it provides an $O((|V| + |E|) \log |V|)$ *continuous propagation* mechanism with experimental validation on downstream update propagation. To address belief change in non-classical logic, existing work provides a logico-algebraic investigation of AGM belief revision based on the logic of paradox (LP) [29]. By defining a concrete belief revision operator for LP, this study proves its compliance with a generalized version of the traditional AGM postulates. Furthermore, it examines the applicability of the classical Levi and Harper identities within a paraconsistent framework, showing that a generalized Levi-type identity yields paraconsistent-based revisions fully compatible with AGM postulates. Ultimately, this demonstrates that once lifted to an appropriate level of generality, the classical AGM framework continues to serve as a regulative ideal for paraconsistent-based epistemic operators. Credibility-limited revision [19] relaxes AGM by restricting revision to a trusted sub-language; NTM's threshold-based mechanism is conceptually similar but operates on continuous belief scores rather than syntactic restrictions.

2.3 LLM Agent Memory Systems

Recent systems address the memory problem at different abstraction levels. To our knowledge, no prior agent-memory system operationalizes partial contradiction as *continuous* downstream belief attenuation, although concurrent 2026 work has begun to measure—and to tackle—the underlying stale/superseded update problem directly (below). Fig. 2 (left panels) summarizes the empirical gap that motivates this work.

OS-tiered memory. MemGPT [6] and Letta [8] treat memory as an operating-system resource. The LLM acts as the OS kernel, issuing read/write/page-out calls to manage Core (context window), Recall (vector-retrieved), and Archival (long-term) tiers. When a fact changes, the agent must explicitly detect the conflict and re-derive affected conclusions; there is no automatic dependency tracking or propagation. This places the full burden of consistency maintenance on the LLM's reasoning at inference time—precisely the cost this work aims to reduce.

Extraction-based flat memory. Mem0 [7] extracts atomic facts via LLM calls and stores each as a versioned vector embedding. Retrieval is semantic (cosine similarity), but with no explicit dependency graph an update to a fact cannot trigger re-evaluation of the conclusions derived from it; versioning tracks fact-level changes without propagating them.

Temporal knowledge graphs (TKGs). They store relational triplets with temporal information, which have attracted increasing attention in numerous downstream applications. Despite the substantial progress made by existing studies on TKG forecasting, they primarily investigate the task on uni-modal TKGs, whereas many knowledge graphs are dynamic and multi-modal [30]. Graphiti (Zep) [9] constructs Neo4j-based temporal knowledge graphs with bitemporal versioning (separate system-time and valid-time dimensions) and supports three-way hybrid retrieval (BM25 + vector + graph traversal). When an entity or relation is updated, Graphiti creates a new temporal snapshot of the affected subgraph. However, downstream nodes that were inferred from the updated node are not automatically flagged for re-evaluation; the agent must independently re-query and re-derive. The propagation mechanism, if present, is binary (node present vs. superseded) rather than continuous.

Multi-graph architectures. MAGMA [13] decomposes memory into four orthogonal graphs (semantic, temporal, causal, entity) and routes retrieval through a strategy-guided traversal of the most relevant subgraph. This improves retrieval precision for complex multi-hop queries but does not address update propagation across subgraphs. A-MEM [14] organizes memory as a Zettelkasten—a self-organizing network of linked notes whose contextual representations are refined as new information arrives. The link structure captures inter-note associations, but the design maintains no explicit premise–conclusion dependency graph: a change to a premise does not cascade a re-evaluation of the conclusions derived from it.

Stale memory and conflict detection. Recent work has begun to address the stale-memory problem

Table 1: Architectural comparison of LLM agent memory systems along the update-propagation axis. "Continuous belief" means a fact's validity is a graded $[0, 1]$ score rather than a binary present/stale flag; "downstream propagation" is what happens to conclusions derived from a fact when that fact changes. This is an architectural, not benchmark, comparison (Sec. 2.3).

System	Dependency graph	Continuous belief	Downstream propagation
MemGPT / Letta [6, 8]	none	no	manual LLM re-derivation
Mem0 [7]	none	no	none (versioning only)
Graphiti (Zep) [9]	temporal KG	no	binary (present/superseded)
MAGMA [13]	multi-graph	no	none (retrieval-only)
A-MEM [14]	link network	no	none (no dependency cascade)
STALE / CUPMem [22]	topology	no	discrete (KEEP/STALE/REPLACE)
MemTX [23]	transaction log	no	cascading rollback
NTM-BP (ours)	probabilistic DAG	**yes** ($[0, 1]$)	**continuous decay**

directly. STALE [22] introduces a benchmark for evaluating implicit conflict resolution in agent memory, revealing that current LLMs struggle to recognize when a later observation invalidates an earlier belief without explicit contradiction. CUPMem, their prototype, proposes write-side state adjudication and topology-triggered belief propagation, but their propagation is discrete (KEEP/STALE/REPLACE) rather than continuous. MemTX [23] introduces transactional belief commits with cascading rollback, providing stronger consistency guarantees than our snapshot-based CoW approach. These works confirm that the stale-memory problem is actively studied; our contribution is to provide a continuous impact propagation mechanism that can serve as a numeric backbone for such systems, reducing the binary STALE/ACTIVE dichotomy to a graded belief score.

Concurrent 2026 evaluations and write-side systems. Three recent preprints are directly adjacent. MEME [24] builds an evolving multi-entity memory benchmark with explicit dependency (*cascade*) and absence-inference tasks and finds that current memory systems collapse on precisely these downstream-propagation cases, empirically supporting this paper's motivation. Supersede [25] isolates and provides a trainable environment for the memory-update gap—the "superseded value" failure mode we observe as supersede-strengthening. The plug-in controller of Chen et al. [26] learns when to accept or reject an incoming write, acting on the *write* path. All three either measure the problem or intervene on the write side; none propagates a continuous, magnitude-proportional decay downstream through a dependency graph, which is our specific contribution.

The common gap. Across all architectures surveyed, engineering effort concentrates on the *write path* (how facts are stored, versioned, and indexed), while the *update-propagation path* is treated as an all-or-nothing decision delegated to the LLM at inference time: when an upstream fact changes, downstream conclusions are ignored, binary-flagged, or manually re-derived, but never *continuously* attenuated in proportion to the change. Table 1 summarizes this comparison at a glance. We acknowledge that a full empirical comparison on shared benchmarks is not yet possible—these systems target different tasks and lack standardized update-propagation evaluations—so our analysis is architectural rather than experimental. Our CoW+BFS baseline captures the essential behavior of the binary strategy (immutable fact versioning plus a BFS traversal that marks every reachable conclusion STALE); NTM replaces this all-or-nothing propagation with continuous, magnitude-proportional belief decay.

2.4 Belief Propagation and Causal Inference

Pearl's [5] belief propagation (BP) algorithm computes marginal posteriors in Bayesian networks by passing messages along graph edges [17]. R. Chang et al. utilize Bayesian belief propagation to infer the responses of perturbation events on molecular traits given a hypothesized graph structure, and define a distance measure between the inferred response distribution and the observed data to assess the "fitness" of the hypothesized causal relationships [31]. In tree-structured graphs, BP converges exactly in $O(d)$ rounds where d is the graph diameter. NTM-BP is *inspired by* this mechanism but does not perform standard Bayesian inference: instead of evidence-to-query posterior computation, we compute source-to-descendant belief *decay*. The multiplicative update $\mathcal{B}_{\text{new}} = \mathcal{B}_{\text{old}} \times (1 - \delta)$ composes naturally along a path, yielding the product form used in our analysis, while the propagation decay coefficient γ serves a role analogous to the damping factor in loopy BP. We therefore describe NTM-BP as a *belief-propagation-inspired confidence attenuation engine* rather than a strict application of Bayesian BP.

2.5 Semantic Gate Mechanisms

LLM-based decision gating—deciding whether to retrieve, act, or re-evaluate—has been explored in retrieval and agent pipelines [11, 16, 18], typically by thresholding a single scalar score to produce a binary decision. We replicated this design as our gate-v1 baseline (Sec. 5). Xiaowen Sun et al. propose a novel SAAG framework, which bases on semantic knowledge obtained from SenticNet and AMR. This framework introduces semantic features into ASTE, enabling the model to process this semantically demanding task by comprehending semantic expressions rather than complex syntactic structures [32].

Our core finding is that scalar-prompt gates exhibit a fundamental **representational coupling** problem. The gate decision for an update depends on at least three orthogonal dimensions: (1) the *magnitude* of the change (is this a reversal, a reinforcement, or a minor fluctuation?); (2) the *relevance* of the update to each downstream conclusion (does this specific update affect that specific conclusion?); and (3) the *directional stance* between the update and each downstream conclusion (does the update align with or oppose the conclusion?). A scalar prompt must jointly encode all three dimensions in a single number, which creates an information bottleneck: the LLM cannot independently adjust its sensitivity along each axis. We observed this empirically as a "whac-a-mole" dynamic during iterative development: improving sensitivity to opposed-conclusion updates (dimension 3) simultaneously caused over-triggering on minor fluctuations (dimension 1), because the scalar threshold could not be tuned independently per dimension.

Our structured decomposition resolves this coupling by factorizing the gate into three separately evaluated sub-decisions (Q1, Q2, Q3 in Sec. 3.3). This factorization is not merely an engineering convenience; it reflects the modular structure of the underlying decision problem (see Sec. 6). Each sub-question addresses exactly one orthogonal dimension, and the decision rule (Eq. (1)) composes them via a simple Boolean formula that preserves the factorization's error-isolation property.

3 Method

Neural Truth Maintenance (NTM) is a framework that generalizes classical TMS from symbolic IN/OUT marking to continuous $[0, 1]$ belief propagation. The term "neural" refers to the LLM-based semantic gate, not to a learned propagation operator. It generalizes classical TMS along three axes: (1) belief states become continuous $[0, 1]$ scores instead of IN/OUT flags; (2) dependency edges carry conditional probability parameters; and (3) propagation replaces binary labeling with multiplicative confidence decay. NTM is realized through two complementary mechanisms: **NTM-BP**, a numeric belief-propagation engine that propagates confidence decay along graph edges without any LLM calls; and the **Structured Semantic Gate**, an LLM-based decomposition that adds semantic precision where numeric decay is insufficient. The two are complementary: NTM-BP serves as a zero-cost numeric backbone (F1 = 0.818 on the full PC-500 set and 0.686 on the held-out expansion set), while the structured gate adds LLM-based semantic analysis for higher precision (F1 = 0.778 on the held-out expansion set) at a modest call cost. Below we formalize the problem, then present each mechanism and their relationship.

3.1 Problem Formulation

Informally, an agent *memory state* $\mathcal{M}$ consists of a set of facts $F = \{f_1, \ldots, f_n\}$ (each a text value with a confidence $c \in [0, 1]$ and a timestamp), a directed acyclic *probabilistic dependency graph* $G = (F, E)$ whose edges $e_{ij} = (f_i, f_j, w_{ij})$ carry a dependency weight $w_{ij} = P(f_j \mid f_i)$ (interpreted probabilistically for convenience; we do not claim calibrated Bayesian semantics), a *belief function* $\mathcal{B} : F \to [0, 1]$ (initialized from each fact's confidence c but updated by propagation), a copy-on-write *version history* $\mathcal{H}$, and a *re-evaluation threshold* τ: a fact is marked `REEVAL_REQUIRED` iff $\mathcal{B}(f) < \tau$. An *update* rewrites one fact f_k to a new value and confidence, preserves the old version in $\mathcal{H}$ (CoW), and must satisfy three axioms—*versioning* (versions never decrease), *probabilistic dependency completeness* (every fact reachable from f_k is either recomputed or its propagated impact falls below a truncation threshold $\varepsilon_{\text{trunc}}$, recovering the classical "recompute everything" requirement as $\varepsilon_{\text{trunc}} \to 0$), and *bounded perturbation* (the total L_1 belief perturbation does not exceed that of the binary cascade under the same reachable-node update setting). The full tuple definition and formal axioms are given in Sec. C.1.

The binary STALE baseline. Existing systems—including our CoW+BFS baseline—implement dependency completeness with a *binary belief function* [2, 3]:

$$\mathcal{B}_{\text{bin}}(f) = \begin{cases} 1 & \text{if } f \text{ is ACTIVE,} \\ 0 & \text{if } f \text{ is STALE.} \end{cases}$$

A BFS traversal from the updated fact marks every reachable node as STALE ($\mathcal{B} = 0$), yielding a re-evaluation set of size $|\text{Reach}(f_k)|$. This is complete in the symbolic sense but discards *all* downstream reasoning regardless of whether the update is a full contradiction ($w \approx 1.0$) or a minor fluctuation ($w \approx 0.1$).

3.2 NTM-BP: Numeric Belief Propagation

NTM-BP is a purely numeric engine requiring zero LLM calls. Given a source update at f_k, it propagates belief decay along the probabilistic dependency graph G using a priority queue ordered by decay magnitude. A key design choice is the initial decay δ_{init}: rather than solely relying on the static CPT $P(v_j \mid v_k)$, we also incorporate the *relative* belief change at the source, $\Delta_{\text{source}} = |\mathcal{B}_{\text{old}}(f_k) - \mathcal{B}_{\text{new}}(f_k)|/\mathcal{B}_{\text{old}}(f_k)$ ($\mathcal{B}_{\text{old}} > 0$ for every initialized fact). A saturating scaling $\alpha(\Delta) = \frac{\Delta}{\Delta + \alpha_0}$ (with $\alpha_0 = 0.15$) compresses the raw change into $[0, 1]$, ensuring that minor confidence adjustments (e.g., $0.95 \to 0.90$) produce a smaller cascade than full reversals ($0.95 \to 0.10$). The initial decay is thus $\delta = \alpha(\Delta_{\text{source}}) \cdot P(v_j \mid v_k)$: a conclusion whose value is fully determined by the premise ($P \to 1$) inherits the full source change, while an edge with negligible dependency ($P \to 0$) transmits no decay. Each subsequent hop applies $\delta_i = \delta_{i-1} \cdot \gamma \cdot P(v_j \mid v_i)$ (Algorithm 1).

The multiplicative decay encodes that influence attenuates with graph distance: after d hops, the effective decay is a product of d factors, each bounded by $\gamma \cdot P(\text{child} \mid \text{parent})$. The saturating scaling $\alpha(\Delta)$ ensures that the cascade magnitude is proportional to the source change—a 0.05 confidence reduction triggers a far smaller propagation than a 0.85 reversal—eliminating the "identical cascade for any update" pathology of pure CPT-based decay. The truncation threshold (0.01) prevents propagation from traversing irrelevant long-range paths. The default hyperparameters ($\gamma = 0.7$, $\tau = 0.7$, $\alpha_0 = 0.15$) are tuned on the held-out development set and validated on Expansion-500 (see Sec. 5.3).

Key semantic difference from the binary baseline. NTM-BP marks nodes as `REEVAL_REQUIRED` rather than STALE. A stale node is presumed invalid and must be recomputed; a re-evaluation-required node merely has insufficient confidence and *may* be recomputed depending on cost-precision tradeoffs.

Optional extension (CO engine). Fig. 2 below also plots a third engine, **CO** (*Co*-relation). CO is an *optional, relation-aware extension* of NTM-BP—not a core contribution of this paper—that scales propagation by the semantic relation carried on each edge (`supports` $\times 1.2$, `contradicts` $\times 1.3$, `neutral` $\times 0.5$). Where edges carry no relation labels (as in PC-500 and SynDep), CO reduces exactly to NTM-BP; we evaluate its relation-aware variants in Sec. 5.5. Readers uninterested in the extension can skip every CO reference without loss of continuity.

Algorithm 1 NTM-BP: Belief Propagation for Memory Updates

Require: Memory $\mathcal{M} = (F, G, \mathcal{B}, \mathcal{H}, t, \tau)$, source fact f_k, new value v', new confidence c', decay $\gamma \in (0, 1)$, scaling $\alpha_0 = 0.15$
Ensure: Updated memory $\mathcal{M}'$

$\mathcal{B}_{\text{old}} \leftarrow \mathcal{B}(f_k)$ `// capture pre-update belief`
$\mathcal{H} \leftarrow \mathcal{H} \cup \{(f_k, v_k, c_k, t)\}$ `// CoW versioning`
$v_k \leftarrow v'$; $c_k \leftarrow c'$; $\mathcal{B}(f_k) \leftarrow c'$
$\Delta \leftarrow |\mathcal{B}_{\text{old}} - c'| / \mathcal{B}_{\text{old}}$
$\alpha \leftarrow \Delta / (\Delta + \alpha_0)$ `// saturating source-change scaling`
$Q \leftarrow$ PriorityQueue() `// ordered by` δ `descending`
visited $\leftarrow \{f_k\}$ `// mark visited on enqueue (matches the implementation)`
for $f_j \in G$.children(f_k) **do**
    $\delta \leftarrow \alpha \times P(v_j \mid v_k)$ `// decay proportional to dependency strength`
    if $\delta > 0.01$ **then**
        visited.add(f_j)
        Q.push$((f_j, \delta))$
    end if
end for
while $Q \neq \emptyset$ **do**
    $(f_i, \delta) \leftarrow Q$.pop()
    $\mathcal{B}(f_i) \leftarrow \mathcal{B}(f_i) \times (1 - \delta)$ `// multiplicative decay`
    if $\mathcal{B}(f_i) < \tau$ **then**
        mark f_i as `REEVAL_REQUIRED`
    end if
    for $f_m \in G$.children(f_i) **do**
        if $f_m \notin$ visited **then**
            visited.add(f_m)
            $\delta_{\text{next}} \leftarrow \delta \times \gamma \times P(v_m \mid v_i)$
            if $\delta_{\text{next}} > 0.01$ **then**
                Q.push$((f_m, \delta_{\text{next}}))$
            end if
        end if
    end for
end while
return $\mathcal{M}'$

3.3 Structured Semantic Gate

While NTM-BP is zero-cost, its purely numeric nature means it cannot distinguish between *confirming* updates ("sales grew 5%" when "sales grew" was already known) and *truly neutral* updates that should not trigger any re-evaluation chain. The Structured Semantic Gate addresses this by decomposing the gating decision into three discrete, independently auditable LLM judgments.

Decomposition. For each updated fact f_k with old value v_{old} and new value v_{new}:

Q1—Change Type:
Is the update a directional change (A: negation, reversal, or the old trend no longer holds), a confirmation or reinforcement (B: same direction but stronger, reconfirmed, or superseded), or a minor fluctuation (C: isolated, episodic event that does not change the overall assessment)? Output: a single letter `A`, `B`, or `C`.

Q2—Relevance:
Given the new situation v_{new}, would knowing it affect the judgment about a downstream conclusion f_j (through direct or reasonable chain-of-consequence influence)? Asked counterfactually: "If this situation holds, would it affect the assessment of that conclusion?" Output: `yes` or `no`.

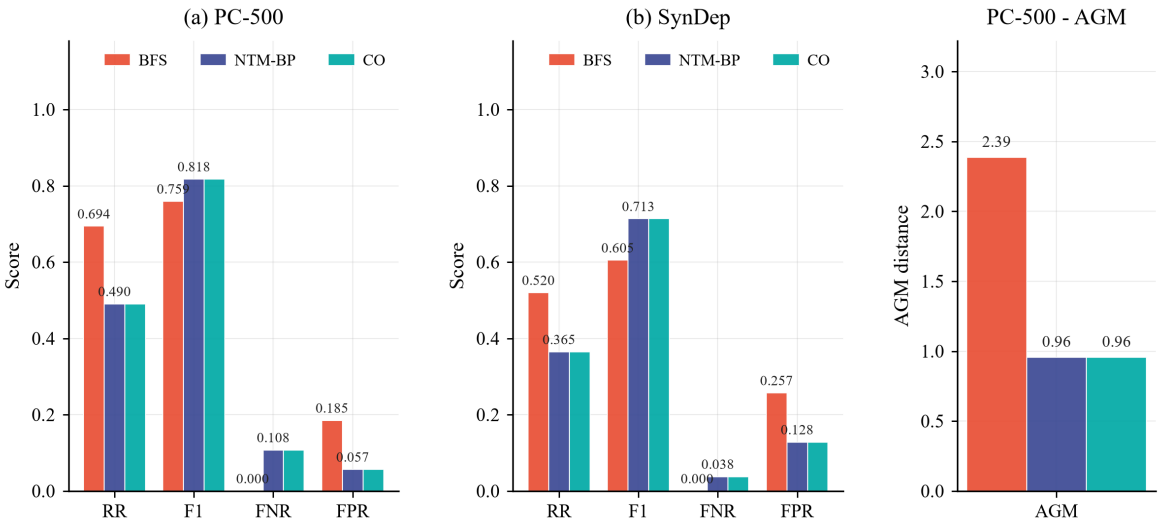


Figure 2: Algorithmic engine comparison on the full PC-500 (500 instances) and SynDep (200 instances) sets: BFS, NTM-BP, and the CO extension across RR, F1, FNR, FPR, and AGM distance (rightmost panel, own scale). Relation types are not annotated on these sets, so CO's edge-type multipliers are inert and CO coincides with NTM-BP; relation-aware CO variants are evaluated on relation-labeled chains (Fig. 7), and the LLM-based gates on the sealed test sets (Table 2). Error bars denote 95% cluster-bootstrap confidence intervals (2,000 resamples).

Q3—Stance:

When Q1 = `B` (confirmation/reinforcement), does the new value align with or oppose the downstream conclusion? `aligned`: the two are consistent or mutually reinforcing; `opposed`: conflict exists such that the new situation weakens the conclusion.

Decision rule. A downstream conclusion f_j is marked for re-evaluation iff:

$$\text{Q2} = \texttt{yes} \;\wedge\; \big(\text{Q1} = \texttt{A} \;\vee\; (\text{Q1} = \texttt{B} \;\wedge\; \text{Q3} = \texttt{opposed})\big). \tag{1}$$

If Q1 = `C` (minor fluctuation), the engine returns immediately after one LLM call per update—no downstream relevance or stance calls are made (Fig. 3). This early-exit path is the primary source of the structured gate's efficiency advantage.

Design rationale. The three-way decomposition was motivated by observing that a single scalar-prompt gate (our gate-v1 baseline) exhibits a "whac-a-mole" dynamic during iterative development: fixing missed opposed-conclusion updates would simultaneously cause over-triggering on minor fluctuations, because the scalar must encode change magnitude, semantic relevance, and directional stance in one number. Decoupling them allows each failure mode to be diagnosed and addressed independently. The counterfactual phrasing of Q2 avoids the "already accounted for" bias that plagued earlier prompt versions.

Relationship between NTM-BP and the Structured Gate. The two mechanisms operate at different levels and are designed to be complementary. NTM-BP is a purely numeric engine: it propagates confidence decay based on the source-change magnitude, edge conditional probabilities, and the decay coefficient γ, without any LLM calls. It handles the common case where a numeric confidence change can be mechanically propagated through the dependency graph. The Structured Semantic Gate operates at the semantic level: it uses LLM judgments to classify the update type, assess per-conclusion relevance, and determine directional stance—dimensions that are orthogonal to the numeric decay and cannot be captured by a single scalar score. In practice, NTM-BP provides a zero-cost fallback with F1 = 0.818 on PC-500, while the structured gate adds LLM-based semantic analysis for higher precision (F1 = 0.778 on the held-out expansion set) at a modest call cost (average 2.87 calls per update). The two can be deployed independently or combined: a cost-sensitive system may use NTM-BP alone, routing to the structured gate only for ambiguous update types (though we show in Sec. 5.5 that naive type-based routing *fails*).

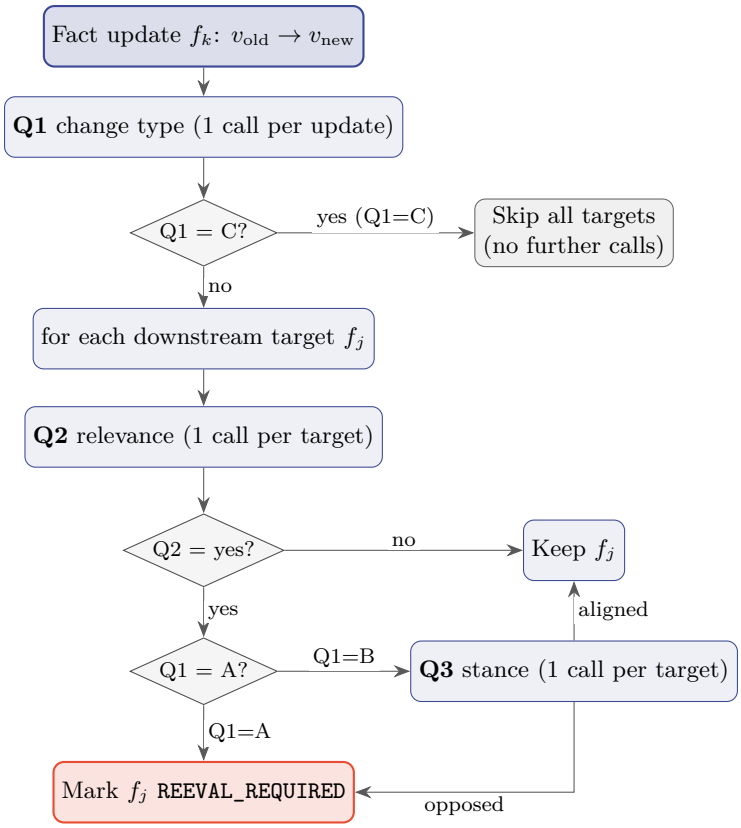


Figure 3: Decision flow of the Structured Semantic Gate. Q1 is asked once per update; if Q1=C all targets are skipped after this single call. Otherwise, for each downstream target: Q2 (relevance) is asked first; on Q2=yes the Q1 verdict decides directly (A → mark), while for Q1=B a third call Q3 (stance) marks only opposed conclusions. Every path thus takes the cheapest sufficient route.

4 Theoretical Analysis

4.1 Cascade Completeness (Baseline)

The BFS cascade is sound and linear-time: by standard BFS analysis (each node and edge visited at most once), it marks every node reachable from the updated source v_k in $O(|V| + |E|)$ time, so no downstream conclusion is missed—but all $|\text{Reach}(v_k)|$ nodes are marked regardless of update magnitude, which is precisely the waste NTM-BP removes. (This is textbook BFS completeness, so we do not dress it as a numbered theorem; the formal statement is in Sec. C.)

4.2 NTM-BP Convergence on Trees

Theorem 4.1 (NTM-BP Convergence on Rooted Trees)**.** *If the dependency graph G is a rooted tree (connected, acyclic, with unique paths from root to all descendants), Algorithm 1 terminates after propagating at most d hops from the updated source, where d is the maximum source-to-descendant path length. The converged belief for any node f_j reachable from f_k is:*

$$\mathcal{B}^*(f_j) = \mathcal{B}_0(f_j) \cdot \prod_{i=1}^{d} \left(1 - \alpha \cdot \gamma^{i-1} \cdot \prod_{m=1}^{i} P_m\right), \tag{2}$$

where d is the path length from f_k to f_j, and P_m is the conditional probability on the m-th edge of the path.

The result is a one-line consequence of the semi-group property of the decay update ($\mathcal{B}_{t+1} = \mathcal{B}_t \times (1 - \delta_{t+1})$ composing as a product along the unique root-to-node path on a tree); the full proof sketch is given in Sec. C.

Remark 4.2 (Loopy graphs: walk-sum view)**.** Real dependency graphs are rarely trees (e.g., mutual reinforcement between "risk is manageable" and "supplier is reliable"). Linearizing the update as $\mathbf{x} = \mathbf{b} + \gamma A\mathbf{x}$, the walk-sum framework [1] gives a unique fixed point whenever $\gamma\,\rho(A) < 1$; on the DAGs we evaluate A is nilpotent ($\rho(A) = 0$), so the condition holds trivially, and our operating point $\gamma = 0.7$ satisfies it for any $\rho(A) < 1/\gamma \approx 1.428$. We verified this *linearized* fixed-point iteration numerically on ring and mutual-reinforcement graphs; the implemented Algorithm 1 uses a visited set and does not itself revisit nodes, so a formal proof for the multiplicative (non-linearized) rule remains a conjecture left to future work. The spectral condition, the numerical protocol, and the results are detailed in Sec. C.

4.3 Precision-Efficiency Tradeoff

We characterize the tradeoff between re-evaluation rate (cost) and accuracy (benefit) that NTM-BP achieves versus the binary cascade.

Proposition 4.3 (Precision-Efficiency Tradeoff)**.** *Let a source update at f_k change its confidence from $\mathcal{B}_{old}$ to $\mathcal{B}_{new}$, producing a normalized source change $\alpha = \big(|\mathcal{B}_{old} - \mathcal{B}_{new}|/\mathcal{B}_{old}\big)/\big(\alpha_0 + |\mathcal{B}_{old} - \mathcal{B}_{new}|/\mathcal{B}_{old}\big)$. For a downstream conclusion f_j at graph distance d from f_k, with edge-wise CPT values $P_1, \ldots, P_d$, the belief after NTM-BP propagation is:*

$$\mathcal{B}(f_j) = \mathcal{B}_0(f_j) \cdot \prod_{i=1}^{d} \Big(1 - \alpha \cdot \gamma^{i-1} \cdot \prod_{m=1}^{i} P_m\Big). \tag{3}$$

Let τ be the re-evaluation threshold. Under the binary cascade, $R_{bin} = 1$ (the node is always marked). Under NTM-BP, $R_{prob} = \mathbb{I}[\mathcal{B}(f_j) < \tau]$, which depends on α, d, the CPT chain, and γ. For deterministic dependence (all $P_i = 1$), every hop transmits the maximal decay $\delta_i = \alpha \cdot \gamma^{i-1}$, so $R_{prob} = \mathbb{I}[\mathcal{B}_0(f_j) \cdot \prod_{i=1}^{d}(1 - \alpha \cdot \gamma^{i-1}) < \tau]$; with a substantial source change (e.g., $\alpha = 0.85$, $\tau = 0.7$) the first hop drops belief to $0.15 \cdot \mathcal{B}_0(f_j) < \tau$, so $R_{prob} = R_{bin} = 1$—deterministic dependencies are always re-evaluated. In the best case (weak dependence, $P_i = P_{\min} > 0$ with $\mathcal{B}_0(f_j) = 1$), the condition for $R_{prob} = 0$ is:

$$\prod_{i=1}^{d} \Big(1 - \alpha \cdot \gamma^{i-1} \cdot P_{\min}^{i}\Big) \geq \tau. \tag{4}$$

Proof sketch. Eq. (3) follows directly from telescoping the recursive decay rule in Algorithm 1: $\delta_1 = \alpha P_1$ and $\delta_i = \delta_{i-1}\gamma P_i$ give $\delta_i = \alpha\gamma^{i-1}\prod_{m=1}^{i} P_m$, and the belief after d hops is the product of the $(1 - \delta_i)$ factors. The deterministic- and best-case claims follow by substituting $P_i = 1$ and $P_i = P_{\min}$ respectively. The full derivation and the empirical calibration against the PC-500 results—including which CPT default (0.5 vs. 0.8) is used where—are given in Sec. C. □ □

4.4 Bounded Belief Perturbation

NTM-BP also gives a quantitative *bounded-perturbation* property relative to AGM-style minimal change; we do not claim it solves the full AGM minimal-change optimization problem. Because the binary baseline zeroes every reachable belief while NTM-BP only multiplies a positive belief by $(1 - \delta_i)$ with $\delta_i \in [0, 1]$, NTM-BP never sets a belief below the binary value, so the total perturbation is bounded by the baseline's:

$$\sum_{f \in F} |\mathcal{B}_{\text{NTM}}(f) - \mathcal{B}'(f)| \leq \sum_{f \in F} |\mathcal{B}_{\text{bin}}(f) - \mathcal{B}'(f)|, \tag{5}$$

with strict inequality whenever any $\delta_i < 1$ (the generic case). This is a one-line observation—multiplicative decay of a positive number stays positive—rather than a deep result, so we state it inline; the formal proposition and proof are in Sec. C.

5 Experiments

We evaluate four methods—BFS cascade (binary baseline), NTM-BP (numeric), gate-v1 (frozen scalar-prompt LLM gate), and the structured gate (our method)—on three datasets under a strict held-out regime. *Naming*

convention: "gate-v1" is the method name used in all tables and figures (also called the "scalar-prompt gate," or simply the "scalar gate," in prose; we write "gate-v1" rather than a bare "v1"). The single scalar prompt was itself refined over internal rounds (v1→v2→v3) during development (Sec. 6.3); all reported numbers use its *final frozen* version, which keeps the method name "gate-v1" to match the shipped cache and is denoted `PROMPT_BUILDERS[v1]` in the reproducibility package; "CO" always denotes the *Co*-relation engine of Sec. 5.5—an optional, relation-aware extension of NTM-BP that coincides with NTM-BP whenever edges carry no relation labels. All LLM-based methods are *frozen before test data generation or annotation*; no prompt iteration touches the test cases. Every LLM call is recorded in a prompt-hash audit cache; evaluation scripts refuse to overwrite existing results (one-shot). Confidence intervals use cluster bootstrap (2,000 resamples, seed 42) resampling at the (template_id, new_value) case level. Because the sealed PC-500 and SynDep test sets are small ($n = 9$ and $n = 18$ cases), their CIs are shown with only two decimal places for tabular consistency and should be read as coarse intervals; the wider expansion-set CIs ($n = 54$) are the more reliable precision indicators.

Metrics. A method's *re-evaluation set* $\hat{R}$ is the set of conclusions it flags for re-evaluation given an update. Comparing against the ground-truth set R^* at the instance level, we report F1 (the harmonic mean of precision and recall of $\hat{R}$ against R^*), the re-evaluation rate (RR = $|\hat{R}|/|$all conclusions$|$, an expected-cost proxy), the false-negative rate (FNR = fraction of R^* not flagged), and the false-positive rate (FPR = fraction of non-required conclusions incorrectly flagged). **Evaluation unit.** Every F1 in this paper is computed *per instance*—one update scored against the ground-truth re-evaluation set R^* of that instance—and then aggregated as an *unweighted (macro) mean* over instances; we do not pool true/false positives globally before forming the ratio. For the expansion set, where 37 of 54 cases have $R^* = \emptyset$, an instance whose correct answer is "re-evaluate nothing" is scored as a perfect match (F1= 1; we treat empty-set agreement as complete agreement) when the method also returns the empty set, and these empty-set matches enter the macro-average on the same footing as non-empty cases. The AGM distance (Fig. 2, rightmost panel) measures the total belief change induced by an update, AGM = $\sum_{f \in F} |\mathcal{B}(f) - \mathcal{B}'(f)|$ (Eq. (5)); lower is better (closer to minimal change).

Datasets. PC-500. 500 instances from 18 annotated cases (6 templates × 3 update values), with per-instance confidence jitter (±0.05). Template-level split yields a sealed test set of 9 cases (249 jittered instances). We distinguish the *sealed* PC-500 split (9 cases, used for final benchmarking, Table 2) from the *full* PC-500 set (18 cases / 500 scenarios, used for exploratory ablations and sensitivity analysis, Tables 5 and 7); every table names the split it reports. Ground-truth re-evaluation sets are manually annotated algorithm-independently (Sec. A); the dual-annotator agreement ($\kappa = 1.0$) applies to the Expansion-500 held-out set described below. **SynDep.** 30 annotated cases (10 templates × 3 update values), 200 jittered instances, covering five topological patterns. Template-level split yields 18 sealed test cases (120 instances). **Expansion-500.** A post-freeze held-out validation set of 54 never-seen cases (×10 jitter = 540 total), generated *after* all prompt iterations finished, inspired by memory-benchmark methodologies [15, 21]. Cases span 18 templates across 12+ domains and five topologies. The complete generation script, the annotation sheets, and the resulting 540-instance dataset are included in the reproducibility package (Sec. A). Two annotators (the author and a blinded advisor, who filled a sheet with all labels removed; Sec. A) labeled every case (Cohen's $\kappa = 1.0$ at both the re-evaluation-set and per-fact levels). Of the 54 cases, 17 (31%) have non-empty ground-truth re-evaluation sets; the remaining 37 (69%) require zero re-evaluation. **Expansion-Balanced.** A second held-out set of 40 adversarial cases balanced 20/20 between non-empty and empty ground truth, designed so that lexical and confidence cues are uninformative (Sec. 5.2). Two annotators (the author and a blinded advisor) labeled every case with perfect agreement ($\kappa = 1.0$) at both the case and per-fact levels.

We organize the experimental analysis around four questions: **(RQ1)** Is NTM effective? **(RQ2)** Why is it effective? **(RQ3)** Which components of the Structured Semantic Gate contribute? **(RQ4)** Where does the method fail?

5.1 RQ1: Is NTM Effective?

Table 2 reports the sealed test results on PC-500 and SynDep.

On PC-500 the sealed split is saturated: all methods, including the binary BFS cascade, tie at approximately 0.85 F1, because in almost every selected case the ground-truth re-evaluation set equals the full downstream

Table 2: Sealed test results (one-shot, frozen prompts). F1 scores with 95% cluster-bootstrap CIs (2,000 resamples).

Method	PC-500 (9 cases)	SynDep (18 cases)
BFS (binary cascade)	0.854 [0.63, 1.00]	0.626 [0.40, 0.83]
NTM-BP (numeric)	0.851 [0.72, 0.96]	0.703 [0.51, 0.86]
gate-v1 (frozen scalar)	0.854 [0.63, 1.00]	0.692 [0.48, 0.88]
Structured (ours)	**0.854** [0.63, 1.00]	**0.807** [0.63, 0.96]

Table 3: Expansion-500 held-out validation (54 cases, 540 instances, dual-annotator labels, frozen methods).

Method	F1
BFS (binary cascade)	0.315 [0.19, 0.44]
NTM-BP (numeric)	0.686 [0.60, 0.76]
gate-v1 (frozen scalar)	0.667 [0.54, 0.78]
Structured (ours)	**0.778** [0.67, 0.89]

set, so binary and selective propagation agree. On SynDep, the structured gate leads by +10.4 points over NTM-BP (0.807 vs. 0.703) and +11.5 over gate-v1 (0.807 vs. 0.692); note that on this split BFS (0.626) is clearly below the selective methods.

Table 3 reports the held-out validation on the 54 never-seen expansion cases.

The structured gate achieves the highest point estimate, leading by 0.092 F1 (9.2 percentage points) over NTM-BP and 0.111 (11.1 percentage points) over gate-v1. A paired cluster-bootstrap (instances nested within the 54 cases) finds the structured-vs-BFS difference (0.463 F1, 46.3 percentage points) significant ($p < 0.001$), but the differences over NTM-BP (0.092 F1, $p = 0.18$) and over gate-v1 (0.111 F1, $p = 0.14$) are *not* significant at $\alpha = 0.05$ with only 54 cases; we therefore do not claim that the structured gate significantly outperforms NTM-BP on this benchmark. *Multiple comparisons.* We report three pairwise tests against the structured gate (vs. BFS, vs. NTM-BP, vs. gate-v1); a Holm–Bonferroni correction across these three comparisons leaves structured-vs-BFS significant (adjusted $p < 0.003$) and does not change the non-significance of the NTM-BP and gate-v1 comparisons, so no claim in this paper rests on an uncorrected p-value.

Stratified analysis: lexically marked vs. unmarked (post-hoc). This split is an *exploratory, post-hoc* analysis: the marked/unmarked stratification was defined *after* observing the structured gate's per-subset performance, so Table 4 is hypothesis-generating rather than a pre-registered confirmatory test, and its subset-level differences inherit the same small-sample caveats as the overall comparison. To understand when each method succeeds, we stratify the 54 cases by whether the new fact value contains an overt negative lexical cue (e.g., "deteriorated," "declined," "strike"). Of 54 cases, 22 are lexically marked and 32 are unmarked. Table 4 reports the results.

On lexically marked cases, the scalar gate achieves the highest F1 (0.955), and the structured gate (0.909) and keyword baseline (0.833) also perform well. The unmarked column, however, must be read with care: 31 of its 32 cases have empty ground truth, so any conservative method scores highly there, and the zero-cost keyword rule (not stratified in this table) in fact reaches F1 = 1.0 on that subset (it correctly skips the

Table 4: Stratified F1 on Expansion-500 by lexical marking. The unmarked split is dominated by empty ground truth (31 of 32 cases), so that column mainly reflects over-trigger behaviour; the keyword baselines are overall-only and unstratified (Sec. 5.5).

Method	Marked (22)	Unmarked (32)
BFS (binary cascade)	0.727	0.031
NTM-BP (numeric)	0.771	0.628
gate-v1 (frozen scalar)	0.955	0.469
Structured (ours)	0.909	**0.688**
Overall-only lexical baselines (not stratified)		
Keyword (text match)	0.833 (overall)	
Keyword+Confidence	0.944 (overall)	

31 empty cases and fires on the single non-empty one), above the structured gate's 0.688; a paired test on the 32 unmarked cases favors the rule ($\Delta = +0.313$, $p < 0.001$, post-hoc; Sec. 5.5). We therefore do *not* read the unmarked subset as evidence that the gate handles unmarked changes better than cheaper rules. The structured gate's advantage over the *scalar* baseline on this subset (ΔF1 $= 0.219$, $p = 0.037$) and its non-significance versus NTM-BP (ΔF1 $= 0.060$, $p = 0.57$) are reported as hypothesis-generating; the regime in which the gate's semantic decomposition does add measurable value is the balanced adversarial set of Sec. 5.2.

Non-empty vs. empty case breakdown. Of 54 cases, 17 require re-evaluation (non-empty ground truth) and 37 require none (empty). On the 170 non-empty instances, BFS, scalar gate, and structured gate all achieve perfect F1 (1.000); NTM-BP scores 0.909. On the 370 empty instances (accuracy = correctly skipping), BFS achieves 0.000 (it re-evaluates everything, so it correctly skips none), NTM-BP 0.584, scalar gate 0.514, and the structured gate 0.676—the highest of the non-trivial methods. These two subsets reproduce the macro-averaged F1 of Table 3: for the structured gate, $(170 \times 1.000 + 370 \times 0.676)/540 = 0.778$. The structured gate thus skips more empty cases than either NTM-BP or the scalar gate; its residual over-trigger rate is confined to borderline semantic changes, the price of the sensitivity that the balanced set of Sec. 5.2 shows is useful when cues are absent.

NTM-BP component ablation. Table 5 shows the contribution of the propagation decay γ versus the binary cascade (Fig. 9).

NTM-BP with $\gamma = 0.7$ reduces the re-evaluation rate by 29.3% relative to BFS (RR 0.491 vs. 0.694) while maintaining F1 above 0.80. Removing the propagation decay ($\gamma = 1.0$) raises F1 to 0.876 but increases RR to 0.569—effectively reverting toward the BFS extreme. The near-zero FNR of $\gamma = 1.0$ (0.005) confirms that propagation without decay is essentially the binary cascade with softer thresholds.

Summary. NTM is effective: the structured gate achieves F1 $= 0.778$ on the held-out expansion set (vs. BFS 0.315), and NTM-BP alone reduces the re-evaluation rate by 29.3% while maintaining F1 $= 0.818$ on the full PC-500 set. The decay coefficient γ is the primary driver of the F1-RR tradeoff.

5.2 Balanced Adversarial Held-out Set

The Expansion-500 distribution is 37/54 empty, so its macro-F1 mostly rewards correctly skipping. To test discrimination in the opposite regime we constructed a second held-out set, Expansion-Balanced, of 40

Table 5: NTM-BP component ablation on full PC-500 (500 scenarios).

Variant	F1	RR	FNR	FPR
BFS (binary cascade)	0.759	0.694	0.000	0.185
NTM-BP ($\gamma = 0.7$)	0.818	0.491	0.108	0.057
NTM-BP ($\gamma = 1.0$, no decay)	0.876	0.569	0.005	0.065

Table 6: EXPANSION-BALANCED held-out results (40 cases: 20 non-empty / 20 empty). "Macro" is the unweighted instance-level macro mean; because the set is balanced, macro and skip-balanced F1 coincide. Labels: dual-annotator, $\kappa = 1.0$.

Method	Non-empty F1	Empty F1	Macro F1
BFS (binary cascade)	1.000	0.000	0.500
NTM-BP (numeric)	0.880	0.850	0.865
gate-v1 (frozen scalar)	1.000	0.750	0.875
Structured (ours)	**1.000**	**0.950**	**0.975**
`kw_text_conf` (zero-cost)	0.650	0.800	0.725

cases balanced 20/20 between non-empty and empty ground truth and designed to separate *semantic* relevance from *lexical* and *numeric* signals. The non-empty cases mostly use neutral surface wording with only a small confidence change—so neither a negative keyword nor a confidence-drop threshold can detect them—while the empty cases deliberately include negative-sounding wording, large confidence drops, and strengthening (supersede-style) updates. Labels were produced by the same written policy as Expansion-500 (Sec. A); two annotators (the author and a blinded advisor) labeled every case with perfect agreement ($\kappa = 1.0$) at both the case and per-fact levels. Table 6 reports the results.

On this balanced set the structured gate attains the highest F1 (0.975), significantly above NTM-BP (paired $\Delta = +0.110$, 95% CI $[0.045, 0.188]$, $p < 0.001$) and the zero-cost keyword rule ($\Delta = +0.250$, CI $[0.10, 0.40]$, $p = 0.001$), though not significantly above the scalar gate-v1 ($\Delta = +0.100$, $p = 0.13$). The keyword rule's F1 falls to 0.725: because the high-impact updates carry no overt negative cue and little confidence change, the rule misses them (non-empty F1 0.65), while it over-fires on negative-sounding but immaterial updates (empty F1 0.80). This is the regime in which the structured gate's semantic decomposition adds measurable value; on the original Expansion-500 distribution, by contrast, the impact signal is largely lexically marked and the rule dominates (Sec. 5.5). We report both regimes side by side rather than the more favorable one alone.

5.3 RQ2: Why Is NTM Effective?

The dependence of re-evaluation rate on the magnitude of the source update, which motivates the entire framework, is illustrated in Fig. 4: BFS maintains a flat, relatively high RR regardless of θ, whereas NTM-BP tracks the impact magnitude; Fig. 8 visualizes the resulting belief states on a small graph.

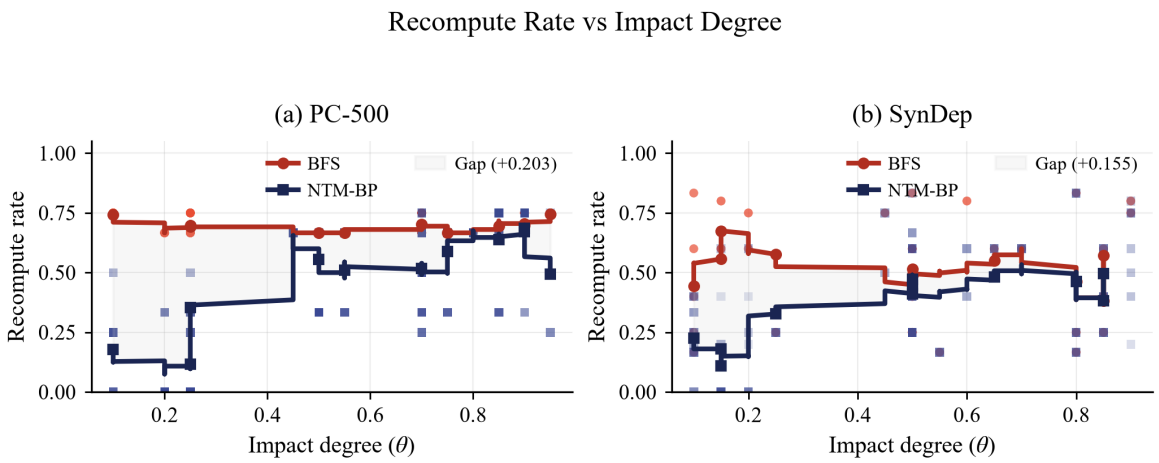


Figure 4: Recompute rate as a function of source-impact degree θ on PC-500 and SynDep. BFS maintains a flat, relatively high RR regardless of θ; NTM-BP tracks the impact magnitude. CO coincides with NTM-BP on these sets (relation types are not annotated), so only NTM-BP is shown.

Per-case win/loss analysis (structured vs. NTM-BP). Out of 54 expansion cases: win 20, lose 9, tie 25. Of the 20 wins, 11 are type-C (minor fluctuation) cases where the structured gate's Q1 early-exit (C→skip) correctly returns after one LLM call while NTM-BP's numeric decay over-triggers; the remaining 9 wins are split across type-A (7) and type-B (2) cases where the relevance path beats numeric decay. This confirms that the structured gate's primary advantage is its ability to *skip* irrelevant minor-fluctuation updates that numeric decay cannot distinguish from genuine changes.

Gamma sensitivity. Table 7 reports the effect of varying γ on PC-500.

The decay coefficient γ exerts smooth, monotonic control over the F1-RR tradeoff. Paired differences are statistically significant for $\gamma \leq 0.8$, indicating that the effect of decay is distinguishable from noise. Figs. 5 and 10 show the corresponding γ–τ sensitivity surfaces.

Scalability. Table 8 reports wall-clock latency per update, with the scaling curves in Figs. 11 and 12.

All methods complete within 20 ms even on 1,000-node graphs, roughly 25–100× faster than a single LLM inference call ($\sim$ 500–2,000 ms).

Robustness to CPT perturbation. Table 9 shows that F1 is essentially flat across the entire noise range (ΔF1 ≤ 0.006). The engine's decisions are driven primarily by the source-change ratio and the γ-attenuated hop structure rather than by the precise edge probabilities.

5.4 RQ3: Which Structured Gate Components Contribute?

Table 10 compares the full structured gate with ablated variants, replayed offline from the frozen prompt cache on the SynDep development and sealed-test splits.

On the sealed test the structured gate reaches 0.807 versus 0.692 for the scalar-prompt baseline (+11.5 points), and the dev-to-test check confirms no leakage (the full gate improves from 0.735 on the dev set to 0.807 on the held-out test set). The per-component ablation, however, does *not* show that each sub-question contributes a measurable accuracy gain: removing Q3 (B always skips) leaves the test F1 unchanged (0.807), replacing the binary relevance judgment with a scalar impact score costs only 0.7 points (0.800), and on the development split removing either *raises* F1 (0.735 → 0.823), i.e., the stance and relevance sub-questions are partially redundant on this distribution and can also over-fire. We therefore attribute the decomposition's measured advantage to the joint three-question design—which beats the scalar baseline—rather than to the marginal value of any single sub-question, and rely on the early-exit cost saving (Sec. 5.3) and the auditability of the logged sub-answers as the operational benefits.

Table 7: Gamma sensitivity on PC-500 (500 scenarios). ΔF1 is the paired bootstrap mean difference vs. $\gamma = 1.0$, with 95% CI. $*$ denotes significance at $p < 0.05$.

γ	F1	RR	ΔF1 vs. no-decay	Significant?
0.5	0.759	0.421	−0.117 [−0.188, −0.043]	$*$
0.6	0.787	0.456	−0.088 [−0.146, −0.031]	$*$
0.7	0.818	0.491	−0.059 [−0.098, −0.017]	$*$
0.8	0.848	0.526	−0.028 [−0.054, −0.004]	$*$
0.9	0.871	0.558	−0.005 [−0.013, +0.003]	no
1.0 (no decay)	0.876	0.569	— (baseline)	—

Table 8: Per-update wall-clock latency (ms) vs. graph size. Measured with the original research build on Python 3.11 / Intel i7-12700H; absolute latencies are hardware- and implementation-specific. `scripts/experiment_scalability_latency.py` benchmarks the shipped dependency-free port.

Method	10	50	100	200	500	1,000
BFS (binary)	0.24	0.75	1.29	3.76	7.52	17.57
NTM-BP	0.24	0.79	1.85	3.29	10.79	18.42
CO (relation)	0.18	0.48	1.58	4.05	7.31	17.64

5.5 RQ4: Where Does the Method Fail?

Failure mode 1: Supersede-strengthening misclassification. All 9 losses relative to NTM-BP are over-triggering cases in which Q1 returns type-A. Most are *supersede-strengthening* updates—e.g., "on-time rate hits an all-time high", "bumper harvest ahead", "additional spectrum approved", "full regression passed", "perfect score from students", "third-party re-inspection all clear", "new school construction begins"—where Q1 misreads supersession as a directional change: "all-time high" and "perfect score" imply the old state has been *superseded*, which the LLM reads as "the old state no longer holds". The remaining two losses are other over-inclusions of class A—a minor single-week dip and a branch-layout adjustment—whose downstream conclusions the annotators judged unaffected. In all cases the structured gate then proceeds down the full relevance path, falsely triggering whole-chain re-evaluation. NTM-BP avoids these errors because its numeric decay is insensitive to semantic framing: a strengthening update with $P(\text{child} \mid \text{parent}) \approx 1.0$ produces near-zero decay and correctly bypasses re-evaluation.

We probed whether this is a *general LLM tendency* (treating supersession as negation) by re-running the frozen Q1 prompt under the reasoning mode of the same model (`deepseek-reasoner`; an auxiliary diagnostic whose raw responses are not part of the shipped cache). On the 10 empty-ground-truth cases labelled type-A by the non-thinking mode (`deepseek-chat`), the reasoning mode agrees on only 4 and reclassifies 6 as B or C (42/54 overall Q1 agreement); these 10 cases are the 9 relative losses plus one in which NTM-BP over-triggers identically. The failure is therefore *partially* shared across inference modes but is not universal:

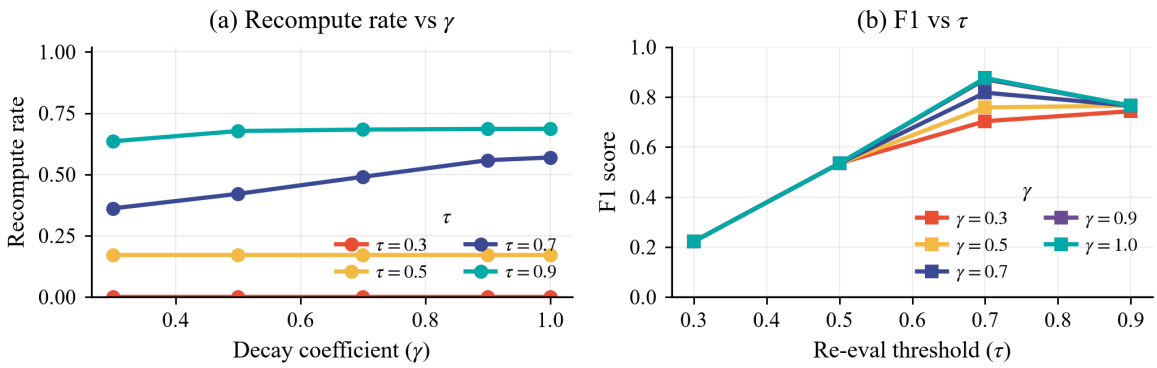


Figure 5: Parameter sensitivity: **(left)** γ vs. RR across multiple τ settings; **(right)** τ vs. F1 across multiple γ settings.

Table 9: NTM-BP F1 on PC-500 under CPT perturbation ($P = 0.8 + \mathcal{N}(0, \sigma^2)$, 10 seeds).

σ	0	0.05	0.10	0.15	0.20
F1	0.819	$0.822_{\pm.003}$	$0.822_{\pm.003}$	$0.824_{\pm.005}$	$0.822_{\pm.005}$

it is a tendency we observe for this model and prompt pair—a stronger reasoning mode reduces it but does not eliminate it—and we do not claim it generalizes beyond these ten cases and this provider. The frozen scalar-prompt gate (gate-v1), using the same `deepseek-chat` model as the structured gate, also fails on 7 of the 9 cases (F1= 0.0), which is consistent with a model/prompt-specific rather than decomposition-specific cause.

Failure mode 2: Keyword baseline outperforms the structured gate. Table 11 reveals that a purely lexical rule augmented with the numeric confidence signal (`kw_text_conf`) reaches F1 = 0.944 at zero LLM cost, *above* the structured gate's 0.778 on the expansion set.

We read the keyword result as a *distributional boundary condition* rather than a refutation. On Expansion-500 the high-impact updates are lexically marked ("worsened", "ambiguity", "missed endpoint"), so a rule that sees both the text and the confidence delta has little left to infer: a paired cluster bootstrap confirms that `kw_text_conf` significantly *outperforms* the structured gate on this set ($\Delta = +0.167$ F1, 95% CI $[0.05, 0.29]$, $p < 0.001$). The structured gate's value is therefore *not* established on this distribution. Two caveats sharpen what that means. First, the set is skip-dominated (37/54 empty), and of the 32 "unmarked" cases 31 have empty ground truth, so any conservative method scores well there; a paired test on that subset also favors the rule ($\Delta = +0.313$, $p < 0.001$). Second, the rule's advantage depends on the impact signal being carried by lexical cues and confidence drops, which need not hold in general: on the balanced adversarial set of Sec. 5.2, where those cues are absent by construction, the rule falls to F1 0.725 and is significantly worse than the structured gate ($\Delta = -0.250$, $p = 0.001$). The gate's distinctive contributions—semantic relevance where cues are absent, and an auditable, decomposed decision whose sub-answers can be logged and contested—are thus real but conditional, and we do not claim them on Expansion-500.

The same zero-cost rule is also competitive on the sealed test sets (Table 12): `kw_text_conf` matches or beats the structured gate on both (F1 0.963 vs. 0.854 on PC-500; 0.851 vs. 0.807 on SynDep) at zero LLM cost. The boundary condition is therefore not specific to Expansion-500: whenever impact-bearing updates carry overt negative cues, a keyword-plus-confidence rule is hard to beat, and the gate's value is confined to update regimes where such cues are absent or misleading, and to auditability.

Table 10: Structured-gate component ablation on SynDep, replayed offline from the frozen prompt cache. The sub-questions are only partially independent: removing Q3 or replacing the binary relevance judgment with a scalar costs at most 0.7 points on the sealed test, and raises F1 on the development split.

Variant	Dev F1	Test F1
Full structured gate (final)	0.735	**0.807**
– Q3 stance removed (B always skips)	0.823	0.807
– Q2 relevance replaced with scalar	0.823	0.800
Scalar-prompt baseline (gate-v1)	0.520	0.692

Table 11: Zero-cost keyword baselines on Expansion-500 (540 instances, cluster bootstrap CI). All baselines make no LLM calls.

Baseline	F1	Re-eval rate
Always skip	0.685 [0.57, 0.80]	0.000
`kw_text`	0.833 [0.72, 0.93]	0.213
`kw_negation`	0.852 [0.76, 0.94]	0.231
`kw_text_conf`	0.944 [0.87, 1.00]	0.333
BFS (binary, all downstream)	0.315 [0.19, 0.44]	0.889

Failure mode 3: Naive type-based routing. Table 13 shows that a naive routing strategy (B→NTM-BP, A→LLM) *fails*: F1 drops from 0.778 to 0.750. The routing rule was frozen only after the results motivating it were observed, so this is an exploratory post-hoc simulation (replayed from the frozen cache, zero new API calls); it is reported here only as a negative result and never as an ex-ante routing evaluation.

All five routing losses occur on Q1→B cases where the numeric path underperforms the structured stance path—NTM-BP cannot distinguish "confirmed and settled" from "confirmed and still moving". This negative result bounds what pure routing can achieve without fixing the underlying Q1 supersede-classification problem.

CO engine: relation-aware propagation. The CO engine extends NTM-BP with edge-type multipliers (`supports` ×1.2, `contradicts` ×1.3, `neutral` ×0.5).

CO outperforms on `supports` (+3.9 F1) and `contradicts` (+5.5 F1) but underperforms on `neutral` (−10.9 F1), reflecting a tension between sensitivity and specificity (Table 14). CO is positioned as an optional extension, not a core component.

Summary. The method fails in three diagnosable ways: (i) supersede-strengthening misclassification (9/54 cases, shared in part by the scalar baseline and by the reasoning-mode probe); (ii) a zero-cost keyword baseline reaches F1= 0.944 on the expansion set, bounding the structured gate's value *on that distribution* (the keyword rule also dominates the "unmarked" subset, which is 31/32 empty); (iii) naive type-based routing fails (0.750 vs. 0.778), showing that path selection cannot substitute for accurate classification.

Table 12: Zero-cost keyword baselines on the sealed PC-500 and SynDep test sets (F1). The split is the same as Table 2 and was not used for prompt tuning; `kw_text_conf` triggers on a negative keyword or a ≥ 0.30 confidence drop.

Method	PC-500 (9 cases)	SynDep (18 cases)
`kw_text`	0.333	0.386
`kw_text_conf`	**0.963**	**0.851**
BFS (binary)	0.854	0.626
Structured gate (ours)	0.854	0.807
NTM-BP	0.851	0.703

Table 13: Routing v0 post-hoc simulation on Expansion-500 (cache replay, zero new API calls). *NOT* ex-ante; reported as a negative result.

Method	F1	LLM calls / update
BFS (binary)	0.315	0.0
NTM-BP (numeric)	0.686	0.0
gate-v1	0.667	2.28
Structured (ours)	**0.778**	2.87
Routing v0 (naive)	0.750	2.24

6 Discussion

6.1 Why the Structured Gate Works

The structured gate has operational benefits beyond the intuitive "divide and conquer" explanation, and—as the ablation of Sec. 5.4 showed—these do not require each sub-question to add a measurable accuracy gain. We identify three complementary mechanisms:

Representational decoupling. A scalar-prompt gate must jointly encode three orthogonal dimensions—change magnitude (Q1), semantic relevance (Q2), and directional stance (Q3)—in a single output token. This creates an information bottleneck: the LLM must simultaneously weigh evidence along all three axes and collapse them into one decision, which empirically leads to the whac-a-mole dynamic observed during prompt development. The structured decomposition removes this bottleneck by assigning each dimension a dedicated inference call, each of which needs only to condition on the update and its own target dimension. The decomposition pays off whenever each isolated sub-question is substantially easier for the LLM than the three-way joint decision it replaces—the regime we observe in practice, where the scalar prompt regressed across three revision rounds while each sub-question prompt converged independently. We stress that this is an *empirical* condition, not a consequence of the factorization alone: the coupled error of a single scalar

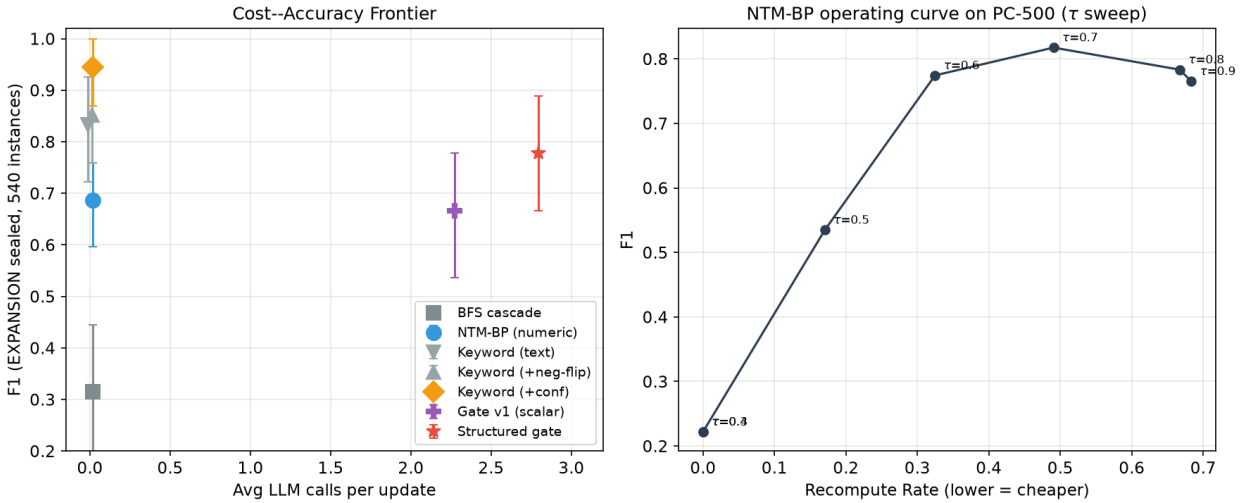


Figure 6: Cost–accuracy frontier. *Left*: F1 vs. average LLM calls per update on the held-out expansion set. *Right*: the tunable operating curve of NTM-BP on PC-500 obtained by sweeping τ.

Table 14: CO engine vs. NTM-BP per relation type (500 scenarios each, oracle labels).

Method / Relation	F1	RR	FNR	FPR
NTM-BP (uniform)	0.818	0.491	0.108	0.057
CO `supports`	0.857	0.543	0.038	0.061
CO `contradicts`	0.873	0.560	0.014	0.062
CO `neutral`	0.709	0.371	0.275	0.053
CO `mixed`	0.813	0.485	0.116	0.058

decision can in principle be smaller than the combined error of three independent ones, and the decomposition helps only because each isolated sub-question is in practice far easier than the joint decision.[1]

Early-exit efficiency. Q1 acts as a cheap negative filter. In the expansion set, 18 of 54 cases (33%) are classified as C (minor fluctuation) and terminate after a single LLM call. The expected number of calls per update under the structured gate is:

$$\mathbb{E}[\text{calls}] = 1 + |D_k| \cdot P(\text{Q1} \neq \text{C}) + |D_k| \cdot P(\text{Q1} = \text{B}) \cdot P(\text{Q2} = \text{yes})$$

where $|D_k|$ is the number of downstream conclusions of the updated fact: Q1 always costs one call; Q2 runs once per downstream conclusion whenever Q1 exits the C path; and Q3 (the stance check) is additionally invoked per conclusion only when Q1=B and Q2 is affirmative. For the expansion distribution audited from the frozen prompt cache ($P(\text{Q1} = \text{C}) = 18/54 \approx 0.33$, $P(\text{Q1} = \text{B}) = 9/54 \approx 0.17$, $|D_k| \approx 2.28$, $P(\text{Q2} = \text{yes}) \approx 0.99$), this yields $\mathbb{E}[\text{calls}] \approx 2.89$, and the exact audited decomposition is $1 + 82/54 + 19/54 \approx 2.87$ calls per update (Q1 once per update, Q2 once per relevant downstream conclusion, Q3 once per stance check), matching the

[1]Writing p_1, p_2, p_3 for the per-sub-question error rates and p_{joint} for the scalar gate's, the decomposed error is $1 - \prod_{i=1}^{3}(1-p_i)$, which expands to $p_1 + p_2 + p_3$ minus higher-order terms; the decomposition is beneficial exactly when this quantity is $< p_{\text{joint}}$, a condition not guaranteed by the factorization alone.

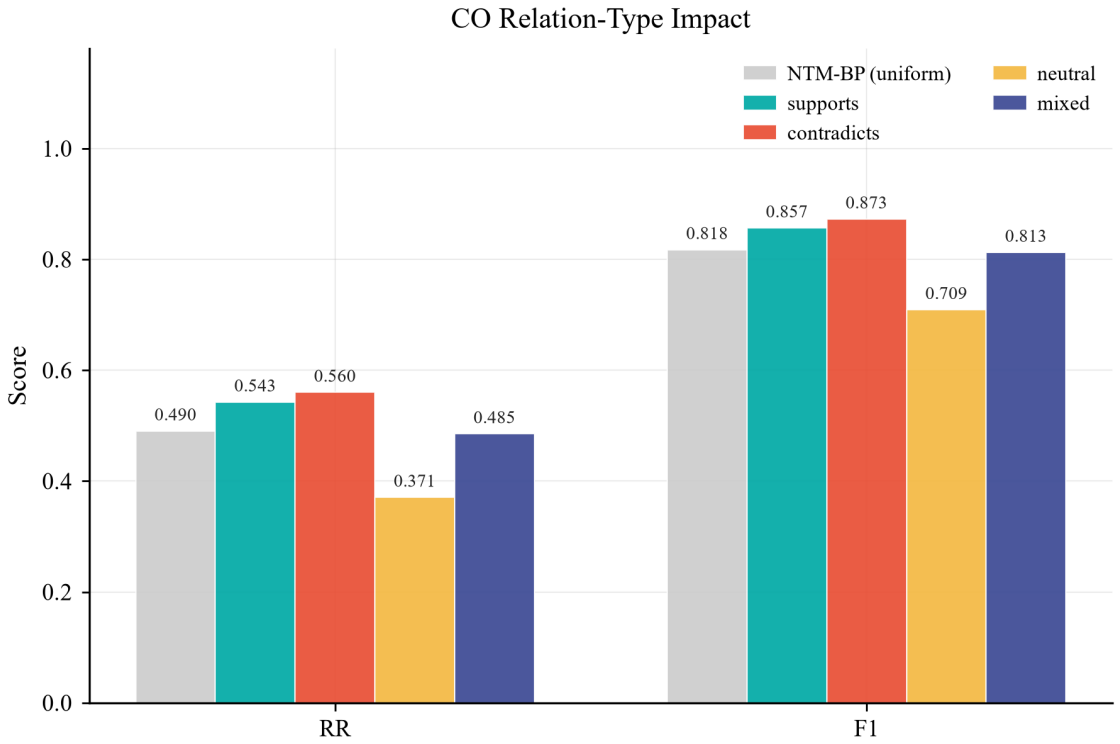


Figure 7: Relation-type impact: NTM-BP vs. CO variants.

measured average of 2.87; the small gap from the closed form is the $|D_k|$-averaging approximation. Relative to an exhaustive three-question evaluation of every update ($1 + 2|D_k| \approx 5.6$ calls), early exit halves the expected call budget.

Error mode isolation. Each failure mode of the structured gate traces to exactly one sub-question. The 9 losses on expansion are all Q1 type-A over-inclusions (7 supersede → A plus 2 other over-inclusions); the 11 Q1 early-exit wins are attributable entirely to the coarse C filter. This one-to-one mapping between sub-question and failure mode is the operational justification for the decomposition: it transforms the gate-tuning problem from "modify a scalar prompt to fix case X without breaking case Y" into "modify sub-question Q_i's prompt to fix class-i errors." Our development experience provides negative evidence for this claim: every attempt to fix a specific failure mode in the scalar prompt (v1→v2→v3) introduced regression in an orthogonal failure class, while the structured decomposition's three substantive changes (Q1: intensify→B, Q2: counterfactual, Q3: content-only) each targeted one error class without observable cross-talk. This mapping is a tuning and auditing property; it does not imply that each sub-question contributes a measurable accuracy gain—the ablation of Sec. 5.4 shows Q2 and Q3 are partially redundant on the tested distribution.

6.2 Design Principles for NTM Gates

The empirical and theoretical results suggest five principles for designing NTM-compatible update gates:

1. **Decompose when dimensions are conditionally separable.** If the gating decision has k orthogonal semantic dimensions, and each dimension can be evaluated independently given the update and higher-priority sub-decisions, then a k-question decomposition reduces both error coupling and average cost. The decomposition is beneficial precisely when the sub-questions are easier to answer individually than jointly—a condition that holds whenever the LLM exhibits the whac-a-mole pathology on the scalar version.

2. **Early-exit on the cheapest negative filter.** Place the coarsest, cheapest-to-answer dimension first.

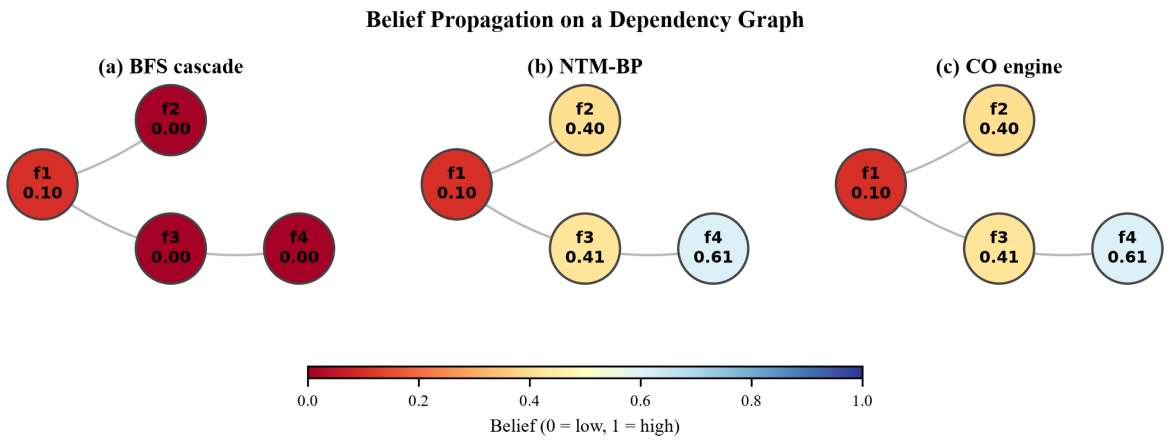


Figure 8: Belief propagation on a small dependency graph. Node colors encode final belief values under the three engines.

Q1 requires only the old and new values (no downstream context), costs one LLM call, and eliminates a third of updates on the expansion set. Any gate design should identify a low-cost triage question that can immediately dismiss a non-trivial fraction of updates.

3. **Maintain a zero-cost numeric path.** NTM-BP's F1 of 0.818 on PC-500 (with decay) provides a deployment-safe fallback: when LLM calls are unavailable or too expensive, the numeric engine alone offers substantially better filtering than BFS (RR reduction from 0.694 to 0.491). The cost-precision frontier (Fig. 6 and Table 13) shows that even a simple hybrid— C→skip, A→LLM, B→NTM-BP— reduces average calls below the structured gate, though at a precision cost.

4. **Calibrate numeric parameters on held-out data.** The propagation decay γ and saturation constant α_0 exhibit smooth, monotonic control over the F1-RR tradeoff (Table 7), but their optimal values depend on the graph topology and CPT distribution. The gamma sensitivity analysis (Sec. 5.3) provides a template: sweep γ on a held-out calibration set, select the value at the elbow of the F1-RR curve, and report paired bootstrap confidence intervals for the chosen operating point.

5. **Make relation-type multipliers asymmetric, with a floor.** The CO engine (Sec. 5.5) shows that edge-level semantic typing modulates propagation quality *asymmetrically*: amplifying `supports` ($\times 1.2$) and `contradicts` ($\times 1.3$) yields F1 gains of $+3.9$ and $+5.5$, whereas dampening `neutral` ($\times 0.5$) over-suppresses propagation and misses genuine re-evaluations (FNR 0.275 vs. 0.108 for uniform NTM-BP). Since amplification helps more than dampening hurts, relation-type multipliers m_r should be floored to prevent excessive suppression—e.g., $\max(0.5, \min(1.5, m_r))$, so no edge type drops propagation below half the uniform baseline. Learning these multipliers (via a small calibrated classifier rather than fixed constants) and integrating CO's edge-type signal with the gate's Q3 stance judgment are natural extensions.

6.3 Limitations and Future Work

Supersede-strengthening misclassification. The primary failure mode—Q1 reading supersede-style strengthening ("all-time high," "perfect score") as a directional change (A), 9 of 54 expansion cases—is characterized in full in Sec. 5.5, together with the evidence that it is a tendency of this model and prompt pair rather than a decomposition artifact: the frozen scalar gate (gate-v1) fails on 7 of the 9 cases, and the reasoning mode of the same model (`deepseek-reasoner`) corrects 6 of the 10 supersede cases but not all; we do not repeat that analysis here. (The scalar prompt received two tracked refinement rounds (v1→v2→v3), but its improvements saturated early—the asymmetry reflects diminishing returns on the coupled scalar path, not unequal effort.) Three fix strategies are available, ordered by invasiveness: (a) refine Q1's definition to

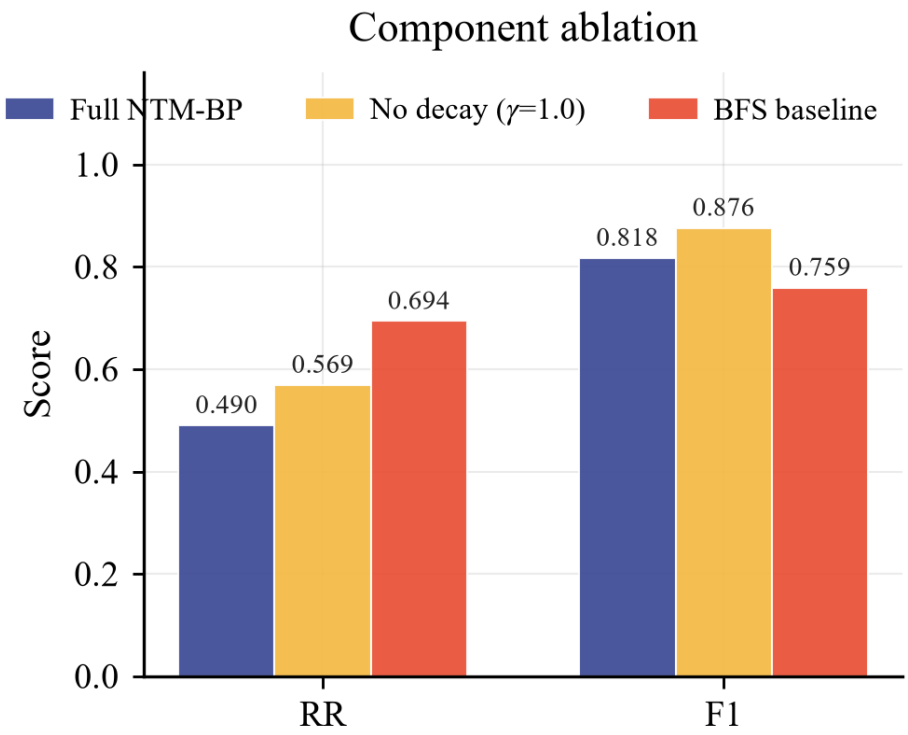


Figure 9: Component ablation: grouped bar chart comparing Full NTM-BP, No-decay ($\gamma = 1.0$), and BFS baseline on RR and F1.

explicitly enumerate "state-surpassed-but-direction-unchanged" as class B (zero architectural change, but requires new held-out data for ex-ante validation); (b) add a Q1.5 sub-question that discriminates supersession from true directional change only when Q1 returns A (adds one conditional LLM call on $\sim 40\%$ of updates); (c) replace the discrete A/B/C classification with a continuous change-magnitude score and threshold it jointly with a semantic direction detector (requires architectural redesign). Strategy (a) is the natural next step, pending a fresh held-out dataset to preserve the one-shot evaluation discipline.

LLM dependency and cost. The structured gate requires LLM calls (audited average 2.87 per update on expansion, corresponding to ~ 850 input tokens and ~ 120 output tokens per call under DeepSeek-V4-Flash, or approximately \$0.0010 USD per update at the provider's published pricing at the time of the experiments (2.87 calls $\times$ (850 input + 120 output tokens); unit prices are provider- and date-specific), making it unsuitable for latency-critical paths. Both the per-call token counts and the \$0.0010 figure are specific to this provider and its published pricing at the time this work was run; they will shift with any change of model, tokenizer, or price schedule, so the cost numbers should be read as an order-of-magnitude illustration rather than a durable constant. The decomposition is not free even relative to scalar gating: the structured gate costs about 26% more calls per update than gate-v1 (2.87 vs. 2.28) on the expansion set, and its point-estimate F1 advantage over gate-v1 is not statistically significant (Table 3); a single-call gate that answers all three dimensions jointly would be the natural cost-matched baseline, and building one is future work. However, the cost structure is distribution-dependent: the expected cost scales with $1 - P(\text{Q1} = \text{C})$, so domains with many minor fluctuations benefit disproportionately from the early-exit path. In the worst case (all updates are type-B with every downstream conclusion relevant), the structured gate costs $1 + 2|D_k|$ calls per update (Q1, one Q2 per conclusion, one Q3 per conclusion)—still bounded by the update's downstream closure, but roughly twice the naive per-conclusion evaluation; the benefit in this regime is error-mode isolation rather than call savings. The zero-cost NTM-BP fallback (F1= 0.818, RR= 0.491 on PC-500) provides a practical alternative: systems can deploy NTM-BP only and accept the 9-point F1 gap on the expansion set (0.686 vs. 0.778; the two are statistically tied at 0.85 on PC-500), or run the structured gate on a sampled subset of

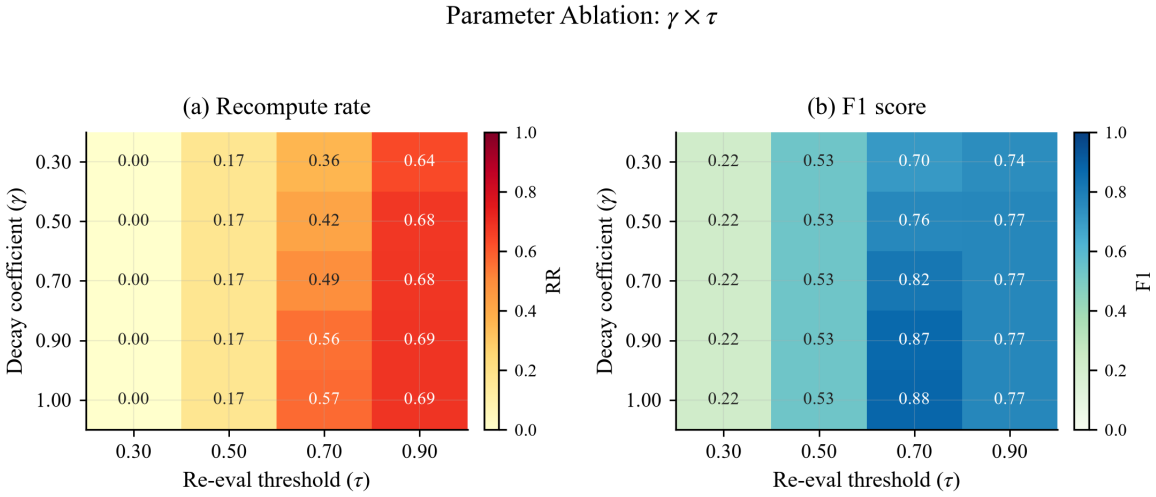


Figure 10: Two-dimensional ablation: **(a)** RR and **(b)** F1 as functions of (γ, τ).

updates for quality assurance while relying on NTM-BP for the remainder.

Lexical boundary condition. The zero-cost keyword baseline of Table 11 outperforms the structured gate on Expansion-500 (F1 0.944 vs. 0.778; paired $\Delta = +0.167$, $p < 0.001$). This is a genuine, distribution-specific boundary of our claim: when impact-bearing updates carry overt lexical cues and the confidence delta is available, a rule needs no semantic inference, and on that distribution we do not claim a gate advantage. The structured gate's value claim is therefore scoped to (i) updates whose impact is *not* lexically marked—on the balanced adversarial set of Sec. 5.2 the gate is significantly better than the rule ($p = 0.001$)—and (ii) auditability: each gate decision decomposes into logged sub-answers that a deployed system can inspect. (On the Expansion-500 "unmarked" subset itself a conservative rule also scores well, because 31 of those 32 cases are empty; we do not read that subset as evidence of semantic skill.) Quantifying the prevalence of lexically unmarked updates in real agent traffic is the key open question that decides how much of the frontier belongs to rules versus LLM gates.

Dependency graph extraction. Our experiments assume the dependency graph G is given. Real deployment must extract `depends_on` edges from unstructured agent dialogue. This introduces two error types: missing edges (false negatives in the graph, causing missed re-evaluations downstream) and spurious edges (false positives, causing unnecessary propagation). Missing edges are the more dangerous failure mode because they break the dependency completeness axiom and can leave genuinely invalidated conclusions unflagged. Spurious edges degrade precision but preserve recall—the safer failure direction for high-stakes applications, where missed invalidation is costlier than unnecessary re-computation. A pragmatic intermediate is to extract edges via structured CoT prompting (where the LLM explicitly cites source facts in its reasoning chain), which yields high-precision edges at the cost of constraining the LLM's output format. Graph extraction quality is orthogonal to the gating problem but determines the upper bound on any downstream gate's recall; we therefore recommend a high-recall, modest-precision extraction strategy. A pilot experiment on CoT-extracted edges (with manual verification as ground truth) is planned as immediate future work to quantify the extraction-to-propagation error cascade.

Non-tree graphs. Theorem 4.1 guarantees convergence only for acyclic graphs. Real dependency graphs may contain cycles (e.g., mutual reinforcement between "delivery risk is manageable" and "supplier reliability is adequate"). The implemented Algorithm 1 maintains a *visited* set, so each node is processed at most once and the algorithm terminates after a single forward pass on any finite graph; on a cyclic graph this pass is well-defined but is not guaranteed to reach a fixed point. More subtly, on a *multi-parent* DAG—acyclic but not a tree, with several incoming paths to the same node—processing each node once means only the first-arriving path contributes to its belief, so the result can depend on traversal order and need not equal a proper aggregation over all paths. The convergence guarantee and calibration above are therefore stated for trees (unique root-to-node paths), and correct multi-path aggregation remains future work. Remark 4.2

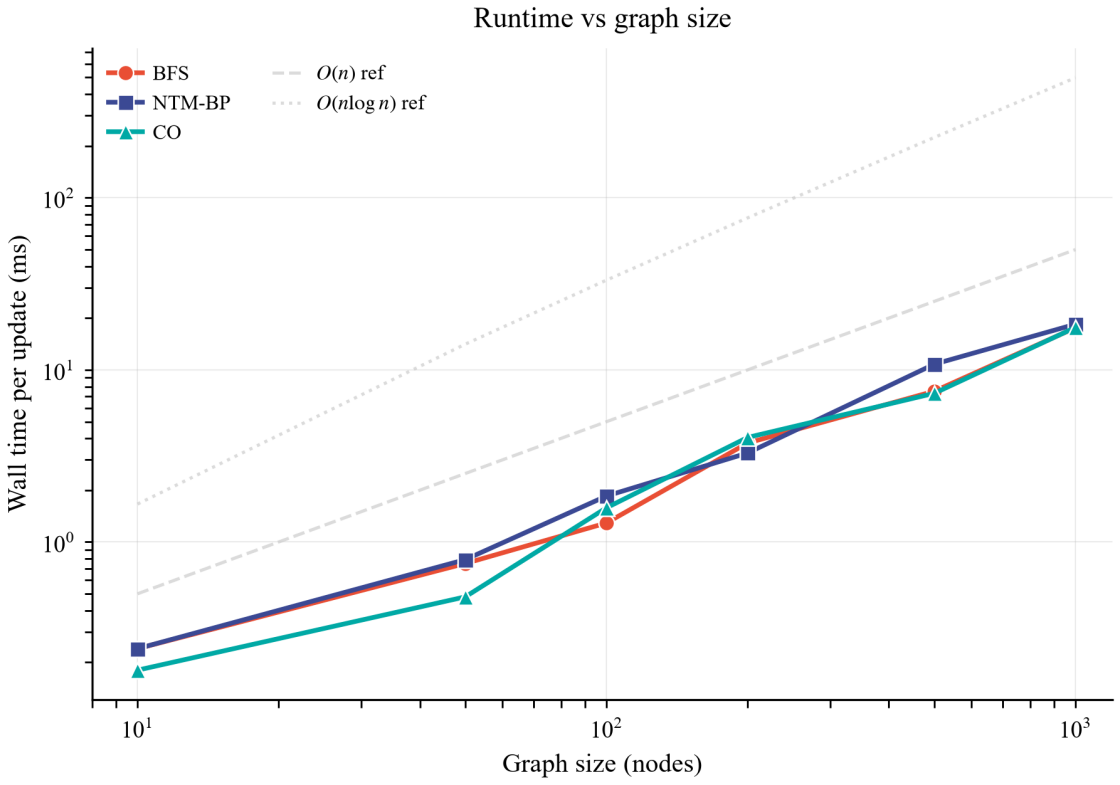


Figure 11: Scalability: per-update wall-clock time vs. graph size (log-log).

analyzes the linearized fixed-point model of the decay update as a theoretical extension of NTM-BP to loopy graphs; extending the implemented multiplicative rule to iterate toward that fixed point, and a formal proof, remain future work.

7 Conclusion

We presented a selective belief propagation framework for truth maintenance in LLM agent memory, replacing binary STALE marking with continuous belief propagation. Two complementary mechanisms—numeric NTM-BP and the structured semantic gate—offer different points on the cost-precision frontier. The structured gate, with its three-way decomposition into change type, relevance, and stance, achieves the highest point estimate among the evaluated non-lexical methods on a fully held-out validation set (54 never-seen cases, dual-annotator labels, $\kappa = 1.0$), with its strongest statistically supported advantage being over the BFS baseline ($+0.463$, $p < 0.001$). On that distribution a zero-cost keyword rule matches or beats the gate, so we do not claim a gate advantage there; on a second, balanced adversarial held-out set whose impact signal is deliberately *not* lexically marked, the structured gate is significantly more accurate than both NTM-BP ($+0.110$, $p < 0.001$) and the keyword rule ($+0.250$, $p = 0.001$), locating the decomposition's value to the regime where surface cues are absent. Its gains come primarily from correctly skipping minor fluctuations (the 11 type-C wins, among 18 type-C cases resolved with a single LLM call), while its losses are concentrated in a single, diagnosable failure mode—an over-inclusive class-A boundary that misreads supersede-strengthening updates as directional changes—shared in part by the scalar-prompt baseline and by the reasoning-mode probe.

Internal validation reveals that NTM-BP is more than a baseline: the saturating source-change scaling eliminates the "identical cascade for any update" pathology; the propagation decay γ yields a smooth F1-RR tradeoff, with paired differences from the no-decay baseline significant for $\gamma \leq 0.8$ ($\gamma = 0.7$: F1= 0.818,

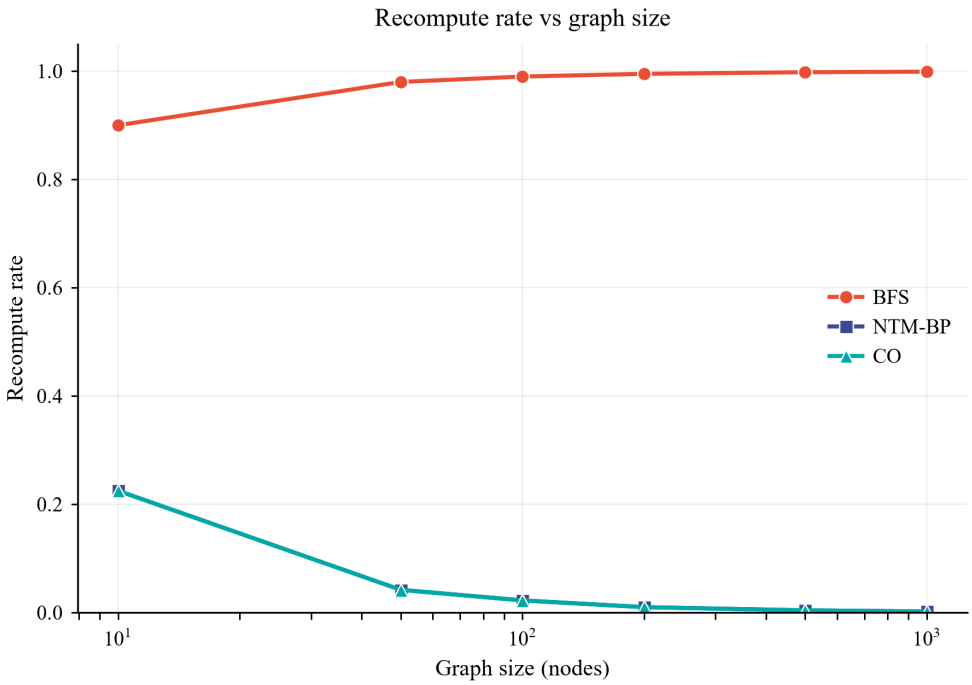


Figure 12: Recompute rate vs. graph size (semilog-x).

RR= 0.491 vs. BFS RR= 0.694); and the engine scales linearly with graph size ($<$ 20 ms on 1,000 nodes). The CO engine further demonstrates that edge-level semantic typing can improve propagation quality (up to +5.5 F1 on contradicts chains), though with a cautionary tradeoff on neutral chains. A naive type-based routing strategy fails (0.750 vs. 0.778), demonstrating that path selection cannot substitute for accurate classification. Future work on Q1 refinement, automatic dependency-graph extraction, and robust relation-type annotation can further close the remaining gap.

Author Contributions

Yuhan Feng — Conceived the research problem from personal experience with LLM coding agents and a biology-inspired analogy of graded (rather than binary) responses; designed the NTM-BP algorithm (continuous belief propagation replacing binary STALE flags) and the Structured Semantic Gate (three-way Q1/Q2/Q3 decomposition). Directed and audited the full Python implementation—specified the architecture, dataset design, and evaluation goals, and audited all code logic, formulas, and results; the code and experiment scripts were developed under the author's direction, with AI providing syntax- and debugging-level assistance. Designed and conducted all experiments, including the main benchmarks (PC-500, SynDep, Expansion-500, CO-Relation), the ablations over γ, CPT and walk-sum conditions, the keyword baselines, and the routing simulation. Performed iterative optimization, debugging, and manuscript writing, and prepared all supplementary materials.

References

[1] D. M. Malioutov, J. K. Johnson, and A. S. Willsky. Walk-sums and belief propagation in Gaussian graphical models. *Journal of Machine Learning Research*, 7:2031–2064, 2006.

[2] J. Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.

[3] J. de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986.

[4] C. E. Alchourrón, P. Gärdenfors, and D. Makinson. On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2):510–530, 1985.

[5] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[6] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*, 2023.

[7] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv:2504.19413*, 2025.

[8] Letta AI. Letta: The memory-first agent framework. Software documentation, https://letta.com, 2026. Accessed August 2026.

[9] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv:2501.13956*, 2025.

[10] Y. B. Park. Graph-native cognitive memory for AI agents: Formal belief revision semantics for versioned memory architectures. *arXiv:2603.17244*, 2026.

[11] W. Yao, S. Heinecke, J. C. Niebles, Z. Liu, Y. Feng, L. Xue, R. Murthy, Z. Chen, J. Zhang, D. Arpit, R. Xu, P. Mui, H. Wang, C. Xiong, and S. Savarese. Retroformer: Retrospective large language agents with policy gradient optimization. In *ICLR*, 2024.

[12] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models' sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. In *ICLR*, 2024. (arXiv:2310.11324.)

[13] D. Jiang, Y. Li, G. Li, and B. Li. MAGMA: A multi-graph based agentic memory architecture for AI agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026, to appear. (arXiv:2601.03236.)

[14] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. In *NeurIPS*, 2025.

[15] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *ICLR*, 2025.

[16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[17] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[18] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[19] S. O. Hansson, A. Fermé, J. Cantwell, and M. Falappa. Credibility limited revision. *Journal of Symbolic Logic*, 66(4):1581–1596, 2001.

[20] J. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

[21] Q. Ai, Y. Tang, C. Wang, J. Long, W. Su, and Y. Liu. MemoryBench: A benchmark for memory and continual learning in LLM systems. *arXiv:2510.17281*, 2025.

[22] H. Chao, Y. Bai, R. Sheng, T. Li, and Y. Sun. STALE: Can LLM agents know when their memories are no longer valid? *arXiv:2605.06527*, 2026.

[23] X. Li et al. MemTX: Transactional belief commit for stateful agent memory. *arXiv:2607.23929*, 2026.

[24] S. Jung, A. Rubinstein, A. Uselis, S. Yun, and S. J. Oh. MEME: Multi-entity & evolving memory evaluation. *arXiv:2605.12477*, 2026.

[25] V. Patel. Supersede: Diagnosing and training the memory-update gap in LLM agents. *arXiv:2606.27472*, 2026.

[26] Z. Chen, S. Dong, C. Shi, P. Wang, S. Wang, C. Shen, and J. Li. The past is prologue: A plug-in controller for selective updates in sequentially evolving LLM memory. *arXiv:2606.31121*, 2026.

[27] T. Frühwirth. Justifications in constraint handling rules for logical retraction in dynamic algorithms: Theory, implementations, and complexity. *Fundamenta Informaticae*, 173(4):253–283, 2020.

[28] D. Fazio and M. Pra Baldi. On a logico-algebraic approach to AGM belief contraction theory. *Journal of Philosophical Logic*, 50(5):911–938, 2021.

[29] M. Carrara, D. Fazio, and M. Pra Baldi. Paraconsistent belief revision: An algebraic investigation. *Erkenntnis*, 89(2):725–753, 2022.

[30] G. Han, W. Chen, X. Zhang, L. Zhang, A. Liu, and L. Zhao. When temporal knowledge graph meets multi-modality: A new perspective for temporal knowledge graph forecasting. *Information Processing and Management*, 63(2):104463, 2026.

[31] R. Chang, J. R. Karr, and E. E. Schadt. Causal inference in biology networks with integrated belief propagation. *Pacific Symposium on Biocomputing*, pages 359–370, 2015.

[32] X. Sun, J. Qi, Z. Zhu, M. Li, H. Pei, and J. Meng. SenticNet and abstract meaning representation driven attention-gate semantic framework for aspect sentiment triplet extraction. *Engineering Applications of Artificial Intelligence*, 139:109625, 2025.

Acknowledgments

Research origin and background. This work grew out of the author's hands-on experience using LLM coding agents on personal projects, rather than from formal training in agent research. The author found the STALE work [22]—whose CUPMem propagates discrete KEEP/STALE/REPLACE states—particularly interesting, and applied its idea in an LLM agent built by the author. In doing so, the author observed that whenever a stored fact was updated, existing downstream conclusions were simply flagged invalid and recomputed in an uncontrolled cascade, inflating inference cost and failing to distinguish minor fluctuations from genuine reversals. Drawing an analogy from biology—where states and responses are typically graded rather than binary—the author proposed the core idea of replacing that binary stale/valid judgment with a continuous 0–1 belief value propagated over a dependency graph. The author then validated the direction's basic feasibility and wrote a first draft of the paper and its formal proofs himself, before refining the exposition with AI assistance and the advisors' guidance.

Advisors. The advisors for this research are Mr. Shu Shi and Mr. Li Zixuan, both teachers at Zhuzhou Nanfang High School. Both are the author's teachers and stand in an advisor–student relationship with the author; neither is a co-author. Mr. Shu Shi provided the computing hardware and runtime environment used in this study, and assisted with technical documentation and writing feedback. During the research implementation stage, he continuously monitored the project's progress, reminded the author of milestone tasks so that the work stayed on schedule and was not abandoned midway, promptly helped the author identify and resolve difficulties encountered, and corrected non-standard practices to ensure that the research process remained scientifically rigorous. Mr. Li Zixuan provided ongoing mentorship and manuscript-writing guidance throughout the project, and assisted the author with paper revision, plagiarism checking, and workflow organization. The advising by both teachers was **unpaid**; the author provided no compensation, direct or indirect, to either advisor, and this research received no guidance or funding from any training or for-profit institution. Neither advisor is a co-author; their role was limited to the support described above, and the core intellectual contributions remain the author's own.

Division of work across the research process. The author directed the research; the breakdown is as follows:

- **Topic selection and problem formulation**: independently proposed by the author from personal experience with LLM agents, with idea-level discussion with the two advisors and AI.
- **Theoretical modeling and proofs**: the author formalized the continuous belief state, the probabilistic dependency graph, and the binary-to-continuous generalization of Doyle's TMS [2], and independently wrote the first draft of the theorems and proofs; AI assisted with wording polishing and derivation checking.
- **Algorithm design**: the author proposed the overall NTM-BP design (multiplicative belief decay, saturating source-change scaling, propagation decay coefficient γ, and priority-queue message passing) and the three-question decomposition of the Structured Semantic Gate; AI offered suggestions at the idea-optimization level, and the author determined and verified the final design.
- **Code implementation and debugging**: the author set the program structure and evaluation goals; the codebase and experiment scripts were developed under the author's direction, with AI assisting at the syntax- and debugging level; the author audited all code logic, formulas, and results item by item.
- **Data acquisition and annotation**: the author designed the dataset templates and served as the first annotator; Mr. Li Zixuan independently completed a blinded annotation without access to existing labels or engine outputs, with perfect agreement ($\kappa = 1.0$). Because the second annotator is also the author's advisor rather than a fully disinterested third party, the perfect agreement is treated as a consistency check on the annotation policy rather than independent external validation.
- **Experimental design and execution**: the research objectives, evaluation goals, and concrete experimental protocol were set and checked by the author; AI assisted with data-analysis tooling and script debugging under the author's direction; the author executed the protocol, reviewed and validated the final protocol, and audited every reported result.

- **Manuscript writing, checking, and optimization**: the author wrote the manuscript body, all figures, tables, pseudocode, and appendices; AI assisted with English polishing, structural organization, argumentation optimization, visual and formatting consistency, and a final pass for typos and internal inconsistencies; the author reviewed and confirmed all of these, and verified the authenticity and citation correspondence of every reference.

Difficulties encountered and resolution.

- *Whac-a-mole coupling in the monolithic scalar-prompt gate*: iterative prompt tuning introduced regressions across disjoint failure modes; resolved by decomposing the reasoning into three orthogonal sub-questions Q1/Q2/Q3, enabling independent convergence.
- *Supersede-strengthening misclassification* ("all-time high" judged as a directional change); identified as a behavioural tendency of this model–prompt pair that the reasoning mode reduces but does not eliminate, documented as the primary remaining failure case, with three mitigation strategies (Sec. 6.3).
- *The zero-cost keyword baseline outperformed the structured gate* (F1 0.944 vs. 0.778); interpreted as a lexical-signal boundary condition.
- *Naive type-based routing underperformed classification* (0.750 vs. 0.778); the negative finding was retained without post-hoc discarding.

AI-usage declaration (per competition rules).

- **Tools and versions**: Doubao (ByteDance); OpenCode, an open-source coding agent, used with the assistant models available to the author—the provider's built-in models and the DeepSeek API; and the DeepSeek API chat model (identifier `deepseek-chat`; the provider's changelog (2026-04-24) lists this alias as corresponding to the non-thinking mode of DeepSeek-V4-Flash) together with `deepseek-reasoner` (the thinking mode of the same model). Exact per-session model identifiers and per-call records are recorded in the submitted chat logs and API records.
- **Stages and purposes**: Doubao for early exploratory direction validation and prototype discussion; OpenCode for code debugging and experiment-script assistance, data-analysis assistance, language polishing, structural organization, and literature-search leads; the DeepSeek API for the LLM semantic-gate evaluations under frozen prompts (temperature 0.1); `deepseek-reasoner` as a reasoning-mode diagnostic probe. AI was additionally used for final manuscript checks (typos and internal inconsistencies), visual/format unification, and structure/argumentation optimization. All cited literature was verified for existence and citation accuracy by the author; AI use was approved by both advisors in advance.
- **Timing and frequency**: approximately 20 tracked assistant sessions between July and September 2026; total LLM API expenditure below $5 USD.
- **Manual verification and boundary**: the author's own contributions are the identification of the problem from personal experience, the core idea, the overall research direction, the first draft of the formal proofs, the experimental design, and the reported analysis. AI assisted with code debugging, data-analysis tooling, language polishing, and structural organization; the author directed this work and manually audited all code logic, formulas, and reported results. Literature authenticity, citation correspondence, and internal consistency were cross-checked by the author. Full chat logs and supporting materials are exported and submitted under "Other Materials" for the committee's verification.

Computing resources. All experiments were executed on the computer and runtime environment provided by Mr. Shu Shi (no discrete GPU); the NTM-BP propagation engines are pure CPU-bound Python. Total LLM-API cost remained below $5 USD.

Responsibility statement. The author is responsible for all content of this report; the data are truthful and reproducible. The author affirms that AI was not used to ghostwrite the main body of the paper, that no fabricated literature was generated or cited, that no experimental data were falsified, and that no paid tutoring or for-profit training was received.

A Reproducibility and Audit Protocol

This appendix documents the experimental discipline and environment configuration that make every number in this paper independently verifiable.

Sealed discipline. Test data were separated by template-level split (seed 20260731) before any prompt iteration. Test cases (PC-500 9, SynDep 18, Expansion-500 54) were never exposed to the prompt developer. The frozen gate-v1 makes zero new API calls on the sealed test set, confirming byte-identity of all prompts.

One-shot evaluation. The evaluation scripts (the sealed-test and expansion-set drivers) refuse to overwrite existing output files. Results are the product of a single execution.

LLM configuration. All experiments use DeepSeek-V4-Flash (model identifier `deepseek-chat`; the provider's changelog (2026-04-24) lists this alias as corresponding to the non-thinking mode of V4-Flash) with temperature 0.1, `max_tokens`= 256, and identical prompts across runs; the one-letter/number gate outputs do not depend on the token cap. The reasoning-mode probe of Sec. 5.5 additionally uses `deepseek-reasoner` with the same frozen prompts. Every LLM call is cached by prompt SHA-256 hash in `llm_gate_cache.json` (954 entries), and no prompt is edited after the freeze. *Determinism caveat.* The frozen cache—not the live API—is the unit of reproducibility: re-issuing an identical prompt to the current `deepseek-chat` endpoint at temperature 0.1 can return a different score than the cached one we observed during the study (e.g., 0.7 live vs. 0.95 cached for one audited prompt), reflecting sampling variance and, possibly, provider-side model revisions behind the alias. All reported LLM-derived numbers are replayed from the committed cache, so they are exactly reproducible from the package, but they should be read as one sampled realization rather than a deterministic function of the prompts.

Bootstrap. Confidence intervals and paired p-values use a cluster bootstrap at the case level (`template_id`||`new_value`), 2,000 resamples, seed 42 (5,000 for the γ sensitivity sweep of Table 7). Resampling draws cases with replacement and takes all ten jitter instances within each drawn case, so instance-level F1 scores are nested within their case; the two-sided p-value is $2\min(\Pr[\Delta^* \le 0], \Pr[\Delta^* \ge 0])$ computed from the resampled paired mean differences Δ^* (structured minus competitor). For the case-level win/loss analysis in Sec. 5.3, each case is represented by its final jitter instance; all aggregate F1 values are instance-level macro means. Jitter applies a ± 5 percentage-point additive perturbation to the confidence value ($c_{\text{jitter}} = c \pm 0.05$, clamped to $[0.05, 0.99]$), small enough that ground-truth labels are preserved—no jitter instance flips the re-evaluation label relative to its parent template.

Data and code availability. A self-contained reproducibility package is provided alongside this submission. It contains the zero-dependency numeric engine (BFS, NTM-BP, CO), the dataset generators, all datasets and dual-annotator labels, the complete frozen prompts, the frozen prompt-hash cache, a SHA-256 manifest of the package, and an offline verifier. With the exceptions of (i) the auxiliary reasoning-mode diagnostic of Sec. 5.5 (whose raw `deepseek-reasoner` responses are not part of the shipped cache) and (ii) the absolute wall-clock latencies of Table 8, which are hardware- and implementation-specific (the package ships the benchmark script), every number reported in this paper is reproduced from the package *without network access or an API key*: the numeric figures and tables regenerate from the deterministic generators (split seed 20260731); the LLM-derived tables (Tables 2 through 4, and Tables 11 through 13) are replayed from the frozen cache and machine-checked by the offline verifier (30/30 checks, zero API calls); and the significance, cost, and scalability audits are recomputed offline by their own scripts in the package. The verifier aborts on any cache miss rather than calling the API, so a pass certifies that the committed cache, datasets, and engine reproduce the reported results. What *is* specified here—the complete frozen prompt texts (Sec. B), all hyperparameters, the dataset construction rules and split (seed 20260731), the metric definitions, the evaluation protocol, the random seeds, and the one-shot execution discipline—remains sufficient to reimplement the protocol independently of the package.

Data statement and annotation protocol. All evaluation data are synthetically generated from templates; they contain no human-subject data, no personally identifiable information, and no sensitive content. Ground-truth re-evaluation labels for Expansion-500 were produced by two annotators following a written policy (judge, from text semantics alone, which downstream conclusions the update materially undermines; minor or reinforcing updates usually yield the empty set). The first annotator was the author; the second was Li Zixuan (advisor), who filled a *blinded* sheet from which all label fields and engine outputs had been removed and who had no access to the first annotator's labels. Agreement was perfect at both the case level and the per-fact level (Cohen's $\kappa = 1.0$); no engine output was used to produce any label. The

same protocol and the same two annotators were used for the EXPANSION-BALANCED set (Sec. 5.2), again with perfect agreement ($\kappa = 1.0$); the second annotator again filled a blinded sheet with no access to the first annotator's labels or any engine output.

Compute resources. All experiments ran on a single consumer-grade Windows workstation (no GPU); propagation engines are pure CPU/Python. Numeric engines (BFS, NTM-BP, CO) process a PC-500 epoch (500 instances) in under one second of wall time; per-update latency on 1,000-node graphs is < 20 ms (Table 8). LLM experiments used the DeepSeek API (`deepseek-chat`): the CO-relation cost–accuracy audit made 32 gate calls ($\approx$ 7,000 tokens, 27 s wall time for both gate variants); the sealed test made 91 new structured-gate calls beyond the frozen cache; all Expansion-500 gate calls were replayed from the prompt-hash cache; and the reasoning-mode probe of Sec. 5.5 made 54 `deepseek-reasoner` calls ($\approx$ 19,000 tokens). The keyword baselines, CPT-noise sweep, and walk-sum verification require no LLM calls and complete in under two minutes. Total LLM spend across the entire project was below $5.

Software environment.

- **Language:** Python 3.9+ (developed/tested on 3.11, 3.14).
- **OS:** Windows 11 / Ubuntu 22.04.
- **Hardware:** Intel Core i7-12700H (14 cores), 32 GB RAM. No GPU required.
- **Dependencies:** `pydantic`$\geq$`2.0`, `openai`$\geq$`1.0`, `numpy`$\geq$`1.24` (bootstrap only), `python-dotenv`$\geq$`1.0`, `pytest`$\geq$`7.0`.
- **LLM API:** DeepSeek-V4-Flash (`deepseek-chat`; the provider's changelog (2026-04-24) lists this alias as corresponding to the non-thinking mode of V4-Flash), temperature = 0.1, max_tokens = 256; the reasoning-mode probe uses `deepseek-reasoner` (the thinking mode of the same model) with the same frozen prompts.
- **Random seeds:** 42 (jitter, bootstrap, CPT noise); 20260731 (case split).

Reproduction principle. All numeric and LLM experiments are deterministic (seed 42; case split seed 20260731). Evaluation scripts refuse to overwrite existing output files, ensuring that reported results are the product of a single execution.

B Full Gate Prompt Texts

Note: The following are English translations of the original Chinese prompts used in all experiments. The deployed prompts are written in Chinese (the language of all evaluation cases). Below we give the complete English translation of each frozen prompt template; placeholders are shown in `<monospace>`.

gate-v1 (scalar impact, frozen).

> In a memory system, a fact has just been updated:
> Old statement: `<old_value>`
> New statement: `<new_value>`
> (Confidence in the new statement is approximately `<new_confidence>`)
>
> The memory contains a downstream conclusion: `<target>`
> `<chain block: direct-dependency note, or list of intermediate conclusions>`
>
> Judge: does this update semantically substantively undermine the validity of that downstream conclusion, such that it needs re-evaluation? Note:
> 1. If the update is semantically essentially unrelated to the conclusion, the impact should be near 0.
> 2. If the update is only a minor fluctuation that does not change the overall prior judgment, the impact should be very low.
> 3. If the update negates or substantially weakens the premise on which the conclusion rests, the impact should be very high.
> 4. Even if the update does not lower the credibility of the source fact itself (e.g., it further confirms an unfavorable condition), as long as it makes the conclusion more untenable, the impact should still be very high.

Output only a single number between 0.0 and 1.0, nothing else.
Impact strength:

Structured gate—Q1 (change type, once per update).

A fact record has just been updated:
Old statement: `<old_value>`
New statement: `<new_value>`
Relative to the old statement, the new statement belongs to which of the following?
A. Directional change: the original situation is negated, reversed, or substantially weakened; or the trend/state described by the old statement no longer holds overall (even if the magnitude of change is small). Note: a single, episodic minor event does not constitute a change of trend/state and should be classified as C.
B. Confirmation or strengthening: confirms, restates, or further solidifies what the old statement already expressed, or shows the original situation developing/intensifying further in the same direction.
C. Minor fluctuation: only a small, episodic change that does not alter the overall judgment (e.g., a one-off minor hiccup or a single mild anomaly); the old statement's overall judgment still holds.
Output only a single letter (A, B, or C), nothing else.
Answer:

Structured gate—Q2 (relevance, per target).

Known situation: `<new_value>`
A certain conclusion: `<target>`
If this situation holds, would it affect the judgment about that conclusion (whether supporting or opposing, direct influence or a reasonable chain of consequences both count)? Answer no only when the two are unrelated in substance (e.g., they merely belong to the same company but no influence chain exists).
Output only "yes" or "no", nothing else.
Answer:

Structured gate—Q3 (stance, per target, only if Q1=B).

Known situation: `<new_value>`
A certain conclusion: `<target>`
Judging by content alone, what is the relation between this situation and that conclusion?
Aligned: the two are consistent or mutually corroborating; the situation holding makes the conclusion more tenable.
Opposed: the two conflict; the situation holding makes the conclusion harder to hold, obstructed, or riskier.
Output only "aligned" or "opposed", nothing else.
Answer:

Parsing defaults (fail-open): an unparseable Q1 answer is treated as A; a Q2 answer whose first characters are not "no" is treated as relevant; a Q3 answer not containing "opposed" within the first four characters is treated as aligned.

C Formal Definitions and Proofs

This appendix collects the formal apparatus that the main text states informally or in summary: the tuple definition of a memory state and its update axioms (Sec. C.1), and the complete proofs and extended derivations summarized in Sec. 4 (Sec. C.2). Nothing here is load-bearing for the empirical claims; it is provided for readers who want the full formal detail.

C.1 Formal Memory Definitions

Definition C.1 (Memory State)**.** *An agent memory state is a tuple* $\mathcal{M} = (F, G, \mathcal{B}, \mathcal{H}, t, \tau)$*, where:*

- $F = \{f_1, \ldots, f_n\}$ *is a set of* facts*. Each fact* f_i *carries a triple* (v_i, c_i, τ_i)*: a text value* $v_i \in \mathcal{V}$*, a confidence* $c_i \in [0, 1]$*, and a timestamp* $\tau_i \in \mathbb{N}$*.*

- $G = (F, E)$ *is a directed acyclic* probabilistic dependency graph. *An edge* $e_{ij} = (f_i, f_j, w_{ij}) \in E$ *carries an edge dependency weight* $w_{ij} = P(f_j \mid f_i)$.
- $\mathcal{B} : F \to [0, 1]$ *is the* belief function*, giving the agent's current confidence in each fact. Note that* $\mathcal{B}(f)$ *is initialized from the fact's confidence* c *but may differ after propagation; we use* c *for the input confidence and* $\mathcal{B}$ *for the propagated belief state.*
- $\mathcal{H}$ *is the* version history *(CoW store), indexed by timestamp.*
- $t \in \mathbb{N}$ *is the global time.*
- $\tau \in [0, 1]$ *is the* re-evaluation threshold*: a fact* f *is marked* `REEVAL_REQUIRED` *iff* $\mathcal{B}(f) < \tau$.

Definition C.2 (Memory Update)**.** *An update* $update : \mathcal{M} \times F \times \mathcal{V} \times [0, 1] \to \mathcal{M}'$ *takes a memory state, a target fact* f_k*, a new value* v'*, and a new confidence* c'*, and produces an updated state satisfying:*

(Versioning)
For all $f \in F$*,* $version'(f) \geq version(f)$*. The old version* (v_k, c_k, t) *is preserved in* $\mathcal{H}$*; the new version* $(v', c', t + 1)$ *is created (CoW).*

(Dependency Completeness, probabilistic)
For every f_j *reachable from* f_k *in* G*, either* $\mathcal{B}(f_j)$ *is recomputed, or the propagated impact on* f_j *falls below the truncation threshold* ε_{trunc} *(Sec. 3.2), in which case the recomputation is skipped with an approximation error bounded by* ε_{trunc}*. In the limit* $\varepsilon_{trunc} \to 0$ *this reduces to the classical requirement that every reachable belief be recomputed.*

(Bounded Perturbation)
NTM-BP's total L_1 *belief perturbation does not exceed that of the binary cascade under the same reachable-node update setting,* $\sum_{f \in F} |\mathcal{B}_{NTM}(f) - \mathcal{B}'(f)| \leq \sum_{f \in F} |\mathcal{B}_{bin}(f) - \mathcal{B}'(f)|$ *(Proposition C.4); we do not claim global minimality.*

C.2 Proofs and Extended Derivations

Proposition C.3 (BFS Cascade Completeness)**.** *For any finite directed graph* $G = (V, E)$*, when a source node* v_k *is updated, the BFS cascade marks all nodes reachable from* v_k *in* $O(|V| + |E|)$ *time.*

Proof. Each node and edge is visited at most once—standard BFS analysis. Soundness (no reachable node missed) is immediate; the cost is that all $|\text{Reach}(v_k)|$ nodes are marked regardless of update magnitude. □ □

Proof of Theorem 4.1 (convergence on rooted trees). On a rooted tree, belief-propagation messages have no cycles. The priority queue processes nodes in order of decreasing accumulated decay; each pop propagates one hop further. After at most d hops the farthest leaf has received the message, so the algorithm terminates. The multiplicative closed form (2) follows from the semi-group property of the decay update $\mathcal{B}_{t+1} = \mathcal{B}_t \times (1 - \delta_{t+1})$, which composes as a product over the unique root-to-node path. □

Loopy graphs: walk-sum detail (extending Remark 4.2). Consider the linearized update model $\mathbf{x} = \mathbf{b} + \gamma A\mathbf{x}$, where A collects the edge-wise decay weights. By the walk-sum framework [1], the unique fixed point $\mathbf{x}^* = (I - \gamma A)^{-1}\mathbf{b}$ equals the series $\sum_{k \geq 0} (\gamma A)^k \mathbf{b}$ if and only if $\gamma\, \rho(A) < 1$, where $\rho(A)$ is the spectral radius of A; the truncation error then decays geometrically at rate $\gamma\, \rho(A)$, and the linearized cutoff-based iteration terminates within $O(\varepsilon/(1 - \gamma\rho(A)))$ of $\mathbf{x}^*$ for cutoff ε. We verified this numerically on ring graphs ($n \in \{3, 5, 10, 20\}$) and a mutual-reinforcement pair embedded in an 8-node chain: for $\gamma \leq 0.9/\rho(A)$ the series converges to machine precision ($< 10^{-11}$) and the cutoff iteration stays within 0.075 of $\mathbf{x}^*$ in ≤ 53 steps, while for $\gamma \geq 1/\rho(A)$ both diverge. The paper's operating point $\gamma = 0.7$ satisfies $\gamma\, \rho(A) < 1$ whenever $\rho(A) < 1/\gamma \approx 1.428$. On DAGs—the structure of all dependency graphs we evaluate—A is nilpotent, so $\rho(A) = 0$ and the condition holds trivially; the numerical tests additionally cover loopy graphs up to $\gamma\, \rho(A) = 0.9$. The cutoff guarantees finite termination on any finite graph but does not establish convergence to the fixed point of the nonlinear update rule; we conjecture that Theorem 4.1 extends to loopy graphs

under the spectral condition above, with a formal proof for the multiplicative (non-linearized) rule left to future work.

Proof of Proposition 4.3 (precision–efficiency tradeoff). Equation (3) follows directly from Algorithm 1's update rule: the initial decay is $\delta_1 = \alpha \cdot P_1$, and each subsequent hop applies $\delta_i = \delta_{i-1} \cdot \gamma \cdot P_i$, which telescopes to $\delta_i = \alpha \cdot \gamma^{i-1} \cdot \prod_{m=1}^{i} P_m$. The belief after d hops is the product of $(1 - \delta_i)$ factors, yielding the closed form. For the deterministic-dependence claim: when all $P_i = 1$, each hop applies the maximal decay $\delta_i = \alpha \cdot \gamma^{i-1} > 0$, so the belief drops monotonically and strong dependencies are always re-evaluated. For the best-case claim: setting $\mathcal{B}_0(f_j) = 1$ and solving $\prod_{i=1}^{d} \left(1 - \alpha\gamma^{i-1} P_{\min}^{i}\right) \geq \tau$ gives the retention condition (4); when it holds, $R_{\text{prob}} = 0$ while $R_{\text{bin}} = 1$. □

Empirical calibration and the two CPT defaults. Two distinct CPT defaults appear in this paper and should not be conflated. All closed-form examples here and the main operating point use CPT = 0.5, the neutral default applied when no calibrated edge probability is available. The CPT-noise robustness experiment (Table 9, in Sec. 5.3) instead perturbs a base of 0.8 to model well-calibrated edges; it tests sensitivity to noise around 0.8, *not* around the 0.5 default used elsewhere. With $\gamma = 0.7$, $\tau = 0.7$, $\alpha_0 = 0.15$, and CPT = 0.5, the gamma sensitivity analysis (Sec. 5.3) provides direct empirical validation. In practical deployment, CPT values can be estimated from historical co-occurrence frequencies, LLM-prompted edge-weight judgments, or calibrated on a small held-out validation set; the framework is robust to moderate CPT variation as long as γ is jointly tuned. The retention condition for a node at distance $d = 1$ is $1 - \alpha \cdot 0.5 \geq 0.7$, which holds when $\alpha \leq 0.6$, i.e., when the relative source change $|\mathcal{B}_{\text{old}} - \mathcal{B}_{\text{new}}|/\mathcal{B}_{\text{old}} \leq 0.225$. For a full reversal ($\alpha \approx 0.85$), the first-hop belief drops to $1 - 0.85 \cdot 0.5 = 0.575 < 0.7$, triggering re-evaluation. At distance $d = 2$, the second-hop decay is $\delta_2 = 0.425 \cdot 0.7 \cdot 0.5 = 0.149$, and the accumulated belief is $0.575 \cdot (1 - 0.149) = 0.489$, remaining below τ. These closed-form predictions align with the observed PC-500 results: at $\gamma = 0.7$ the overall re-evaluation rate is 0.491 (a 29.3% reduction from BFS's 0.694) and F1 is 0.818, confirming that $\tau = 0.7$ selectively marks genuinely affected nodes while retaining those with weak or distant dependencies.

Proposition C.4 (Bounded Belief Perturbation)**.** *The total belief perturbation induced by NTM-BP satisfies the bound (5), with equality only when every reachable node falls below τ and NTM-BP sets its belief to zero (which Algorithm 1 does not do).*

Proof. The binary baseline sets $\mathcal{B} = 0$ for all reachable nodes, yielding a total perturbation of $\sum_{i \in \text{Reach}(f_k)} \mathcal{B}_0(f_i)$. NTM-BP applies multiplicative decay $\mathcal{B}_{\text{NTM}}(f_i) = \mathcal{B}_0(f_i) \cdot (1 - \delta_i)$ with $\delta_i \in [0, 1]$, so $\mathcal{B}_{\text{NTM}}(f_i) \geq 0$ for all f_i. Since $\mathcal{B}_{\text{NTM}}(f_i) \geq 0 = \mathcal{B}_{\text{bin}}(f_i)$ for all reachable nodes and the pre-update beliefs are identical, the absolute perturbation satisfies $|\mathcal{B}_{\text{NTM}}(f_i) - \mathcal{B}_0(f_i)| \leq |\mathcal{B}_{\text{bin}}(f_i) - \mathcal{B}_0(f_i)|$ for each f_i. Summing over all reachable nodes yields (5). Strict inequality holds whenever any $\delta_i < 1$, which is generically the case. □ □